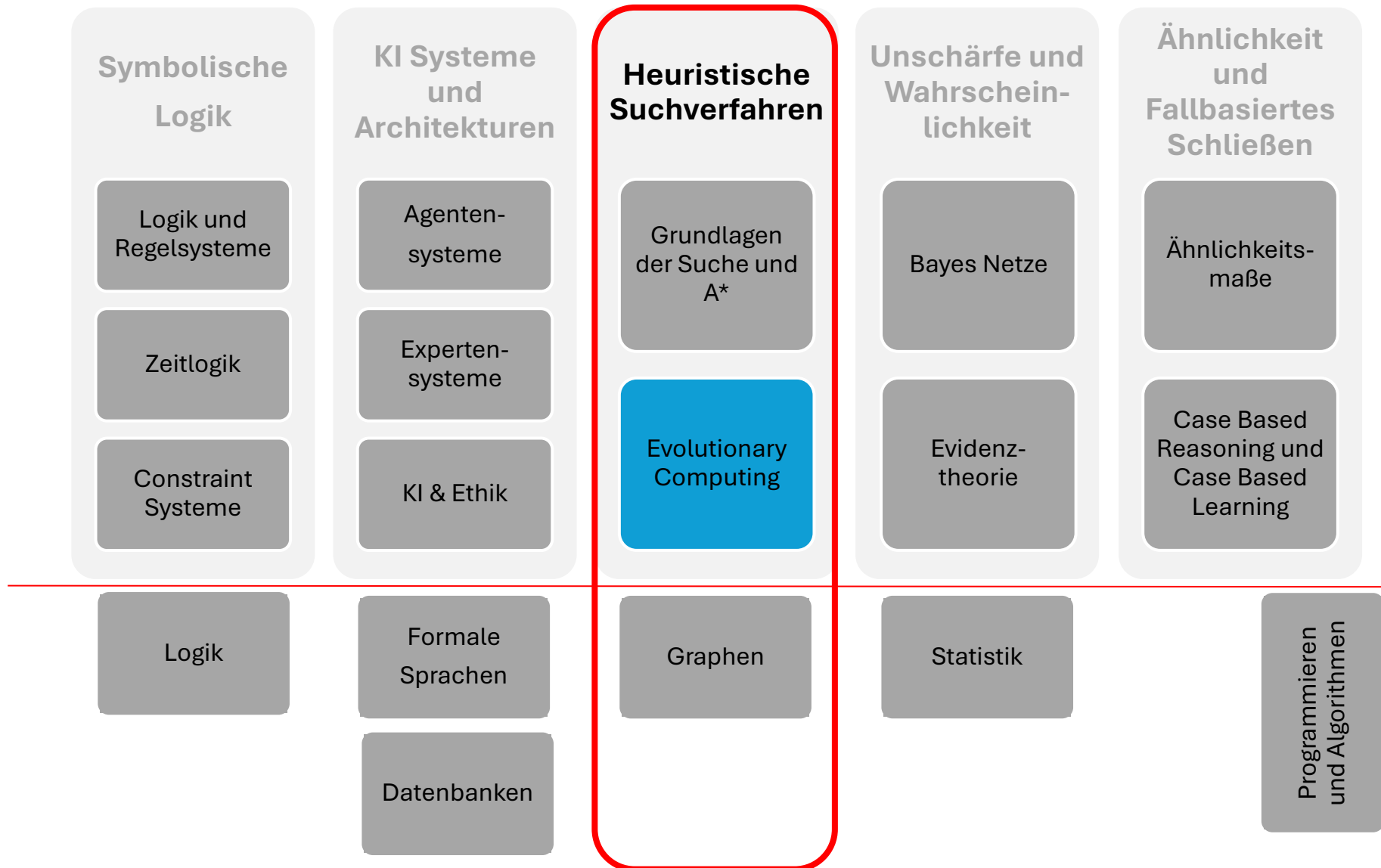


Evolutionary Computing

Teil 1: Einführung mit Beispielen

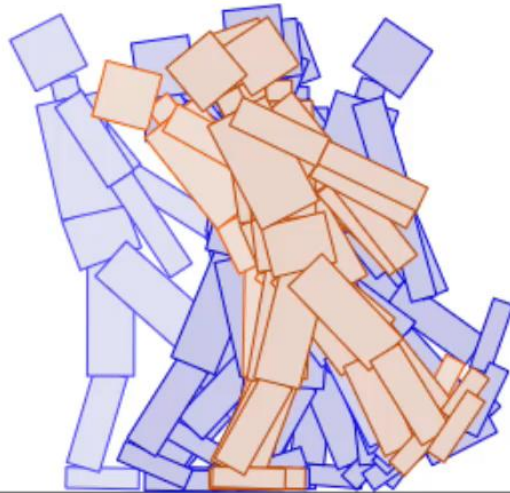




Zunächst einige Beispiele: Genetic Algorithm Walkers

Genetic Algorithm Walkers

"Nice me-and-my-friends-leaving-the-bar-at-2am simulator!"



https://rednuht.org/genetic_walkers/

What the hell is this?

This observational pastime hopes to evolve walking creatures through genetic algorithms.

How is the score calculated?

In order to promote upright walking, each creature receives points based on how high the head is in relation to its feet, and how much it advances while upright. Bonus points are given for each proper step forward.

(Source: Quotes from website)

Current Generation (57)

Name	Score
Bebepo laaapi	205.81
Sedepi laaapi	105.32
Fudipo Horapi	205.84
Bidiqo Iaropi	104.68

Record History

Gen	Name	Score
0	Baeicu Rabeqe	4.28
1	Halouu Lahiou	4.66
2	Bieiou Jabeqe	4.74
3	Bifiri Zirehe	105.42

Controls

Gene mutation probability 10% ▾

Gene mutation amount 50% ▾

Champions to copy 4 ▾

Motor noise 0.5% ▾

Round length Regular ▾

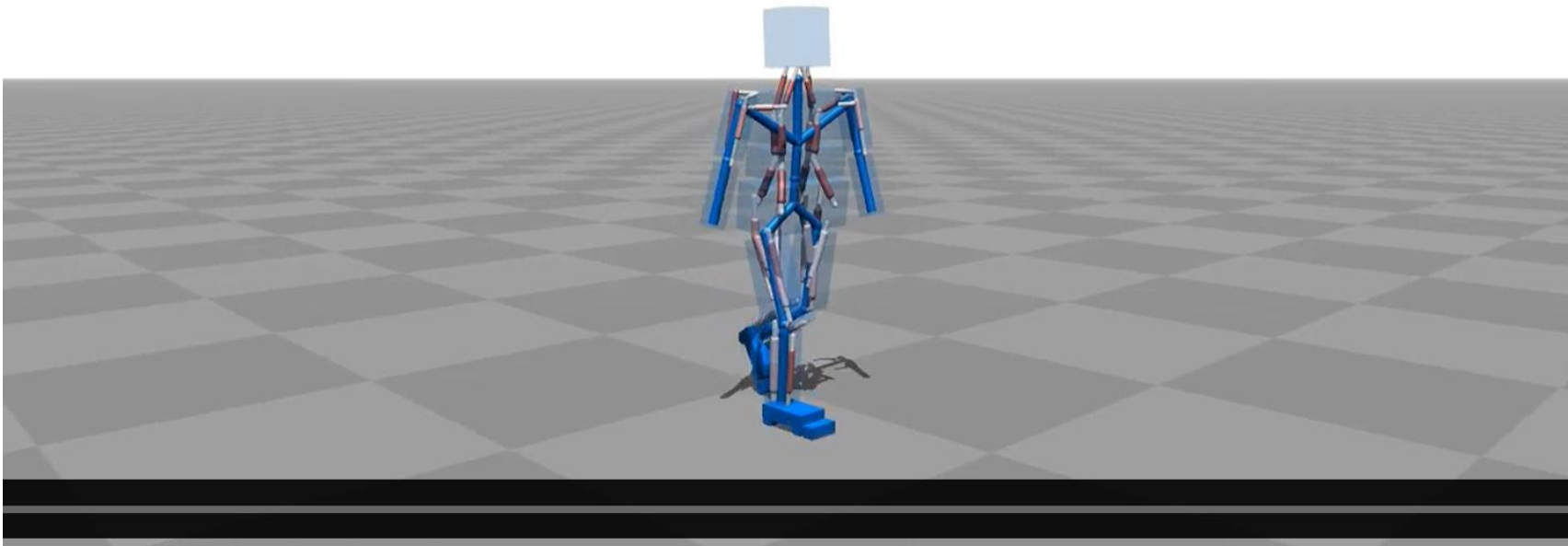
Animation quality 60 fps ▾

About

What the hell is this?

This observational pastime hopes to evolve walking creatures through genetic algorithms.

Simulated Biped



<https://vimeo.com/80822626>

BoxCar 2D

BoxCar 2D

[Home](#) | [Designer](#) | [Best Cars](#) | [Forum](#) | [News](#) | [FAQ](#) | [The Algorithm](#) | [Versions](#) | [Contact](#)

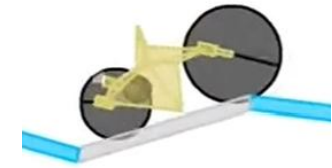
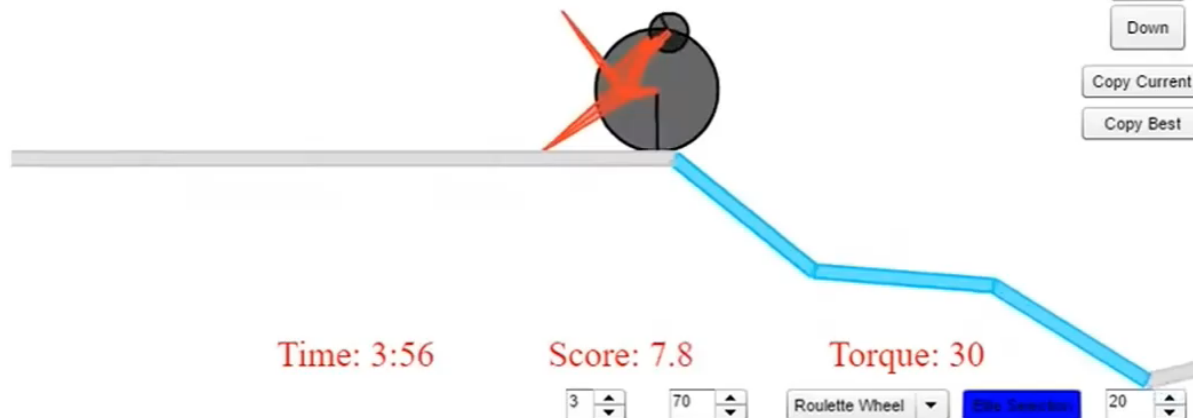
Computation Intelligence Car Evolution Using Box2D Physics (v3.2)

31 fps average
Physics step: 15 ms (68 fps)
23 MB used

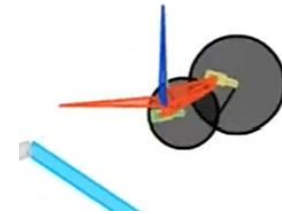
100
Generation: 0 Max Score: 1.6

Car	Score	Time
0	0	0:00
1	0.1	0:00
2	0.3	0:00
3	1.6	0:03

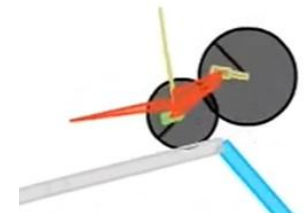
50



Gen 0 Gewinner



Gen 1 Gewinner



Gen 2 Gewinner

<http://www.boxcar2d.com/index.html> (no longer functional)

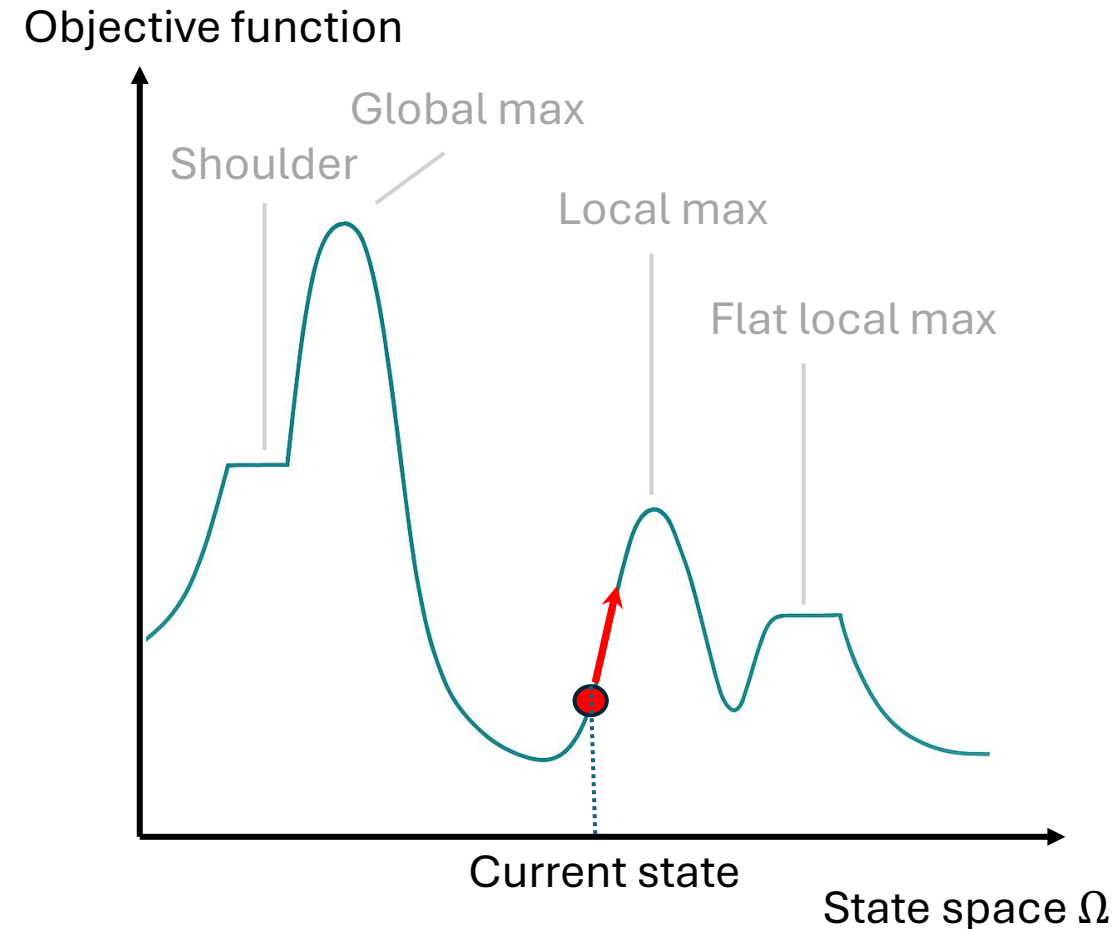
Evolutionary Computing

Teil 2: Theoretische Grundlagen



Optimierungsproblem

- Ein **Optimierungsproblem** $(\Omega, f, >)$ ist gegeben durch einen **Suchraum** Ω , eine **Bewertungsfunktion** $f: \Omega \rightarrow \mathbb{R}$, die jedem Lösungskandidaten einen Güterwert zuweist, sowie eine **Vergleichsrelation** $> \in \{<, >\}$.
- **Problembeschreibung:**
 - **Gegeben:** Optimierungsproblem $(\Omega, f, >)$
 - **Gesucht:** Ein Element $x \in \Omega$, das die Funktion f *global* optimiert
- Dann ist die Menge der **globalen Optima** $\mathcal{H} \subseteq \Omega$ definiert als
$$\mathcal{H} = \{x \in \Omega \mid \forall x' \in \Omega: f(x) \geq f(x')\}$$

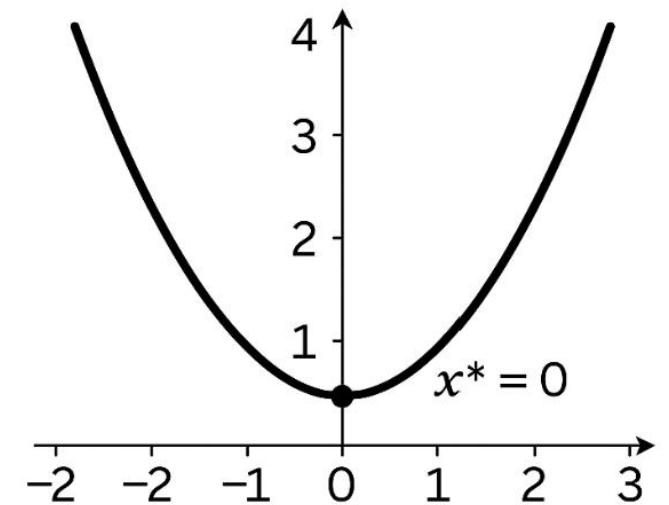


Quelle: Kruse, Held: Evolutionäre Algorithmen, Uni Magdeburg
(http://fuzzy.cs.ovgu.de/ci/ea/ea2013_v01_einfuehrung.pdf)

Prinzipielle Lösungsansätze (1/3)

- **Analytische Lösung**

- *Sehr effizient, aber nur selten anwendbar*
- **Beispiel:** Suche das Minimum der Funktion $f(x)=x^2+0.5$
 - Bilde die *Ableitung*: $f'(x)=2x$
 - Bestimme die *Nullstellen*: $f'(x)=0 \rightarrow x=0$
 - *Zweite Ableitung* prüfen: $f''(x)=2>0 \rightarrow \text{Globales Minimum}$



Prinzipielle Lösungsansätze (2/3)

- **Vollständige Durchforstung** (Brute Force)
 - Sehr *ineffizient* und nur bei *sehr kleinen* Suchräumen anwendbar
 - **Beispiel:** Ein Dieb kann einen Rucksack **maximal 10kg** tragen. Er überlegt, welche Gegenstände er stehlen soll...:

Gegenstand	Gewicht	Wert
Goldbarren	6	30
Silberbarren	3	14
Uhr	4	16
Laptop	2	9
Kamera	1	5

- **Optimale Lösung via Brute-Force:**
 - Laptop + Silber + Kamera = **Gewicht 6kg, Wert 28** **ODER**
 - Gold + Kamera = **Gewicht 7kg, Wert 35 (BESSER!)** **ODER**
 - ...
 - Gold + Silber + Kamera = **Gewicht 10, Wert 49 (OPTIMUM)**



→ $2^5 = 32$ mögliche Kombinationen,
→ Allgemein: 2^N

Prinzipielle Lösungsansätze (3/3)

- **Blinde Zufallssuche**

- *Idee:* Zufälliges Ausprobieren von Lösungen ohne Systematik oder Strategie
- Immer anwendbar, aber meist sehr ineffizient und garantiert kein (globales) Optimum

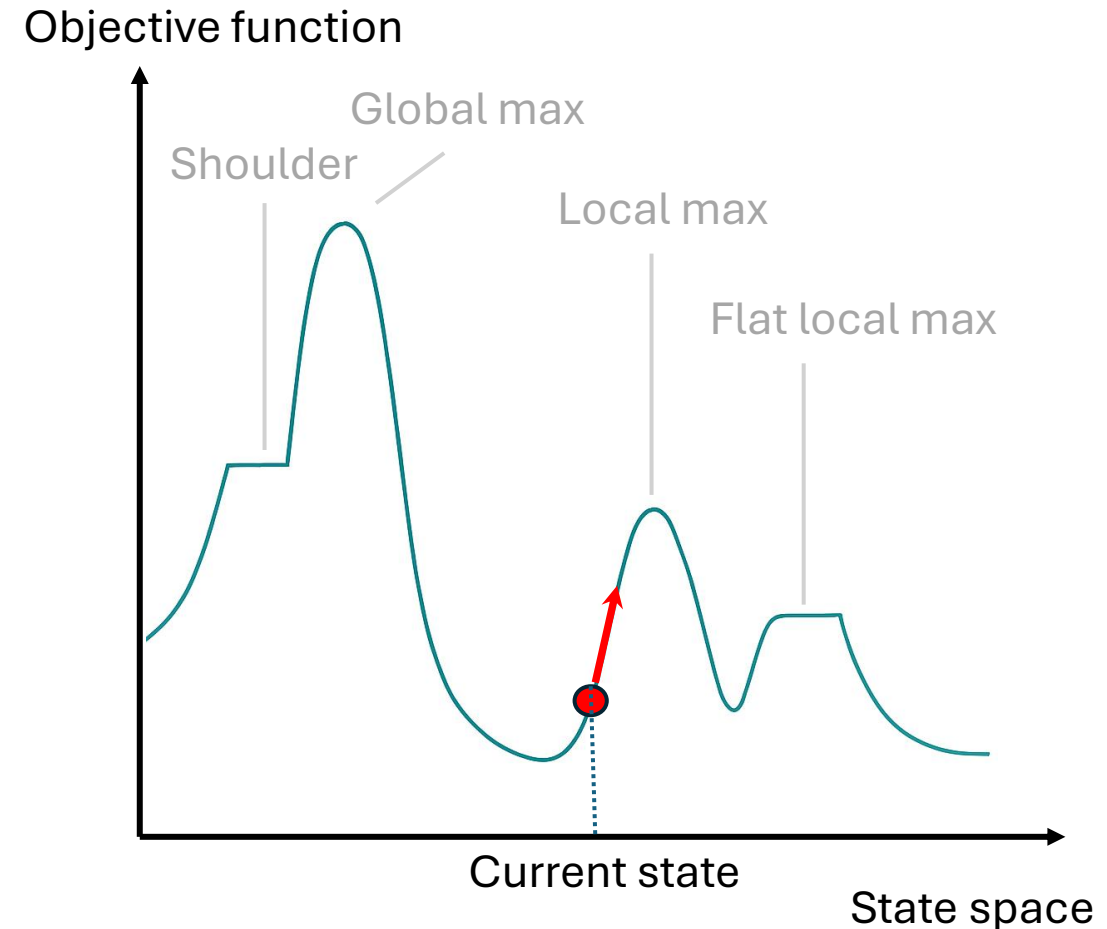
- **Gesteuerte Suche**

- *Voraussetzung:* Ähnliche Elemente aus dem Suchraum Ω haben ähnliche Funktionswerte f
- *Idee:* Gezielt entscheiden, wo man weitersucht; garantiert oft auch kein (globales) Optimum, aber zielgerichteter und damit häufig besser
- *Beispiele:*
 - **Informierte Suche** (Greedy Best, A*)
 - **Lokale Suche** (Hill Climbing, Simulated Annealing)
 - **Evolutionäre Algorithmen**

Quelle: Kruse, Held: Evolutionäre Algorithmen, Uni Magdeburg
(http://fuzzy.cs.ovgu.de/ci/ea/ea2013_v01_einfuehrung.pdf)

Reminder: Lokale Suche

- Sprechender Name: Es werden nur **lokale Nachbarn** vom aktuellen Zustand untersucht
- Meist nur Speichern der **Lösung**, nicht des Pfades zur Lösung
- Meist genutzt für das Lösen von **Optimierungsproblemen** (Maximum/Minimum der Zielfunktion finden)
- Pros/Cons:
 - + Speichereffizient, schnell
 - Lokale Maxima/Minima, Plateaus



Evolutionäre Algorithmen

Heuristik	Problemspezifischer Trick zur Verbesserung der Suche (A*: Luftlinie für kürzeste Wege)
Metaheuristik	Allgemeines, oft domänenunabhängiges Steuerungskonzept (z.B. Simulated Annealing)

Definition Wikipedia:

- „**Evolutionäre Algorithmen (EA)** sind eine Klasse von stochastischen, **metaheuristischen** Optimierungsverfahren, deren Funktionsweise von der Evolution natürlicher Lebewesen inspiriert ist.
- In Anlehnung an die Natur werden Lösungskandidaten für ein bestimmtes Problem künstlich evolviert, EA sind also **naturanaloge Optimierungsverfahren.**“

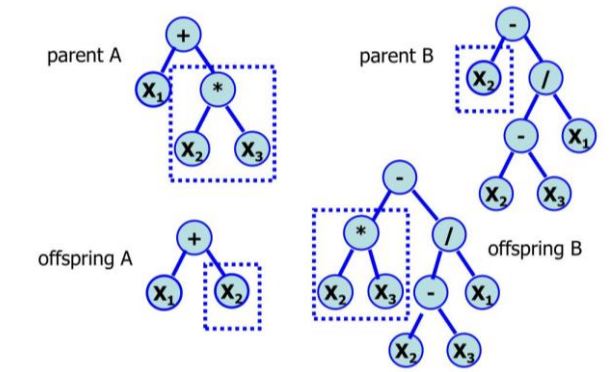


Die Antenne der 2006er [Space-Technology-5](#)-Satelliten wurde mit einem EA entwickelt.

Quelle: [Evolutionärer Algorithmus – Wikipedia](#) (10/25)

Kurze historische Einordnung

Jahr	
~1940-50	Alan Turing erwähnt den Einsatz von genetischen Algorithmen für den Einsatz von Suchverfahren
~1960-70	Mehrere Ansätze werden unabhängig verfolgt, es treten neue Begriffe auf: „ <i>genetic algorithms</i> “, „ <i>evolutionary computing</i> “
~1990	Der Begriff „ <i>evolutionary computing</i> “ setzt sich durch
~2001	<i>Genetic Programming</i> ist als eigenes Verfahren anzusehen – Lösungssuche in einem Baum



Crossover (Source: University of Aberdeen)

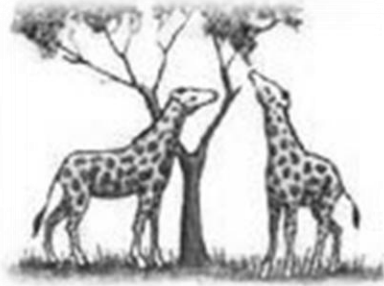
Quelle: Dr. S. Gillner, DHBW Skript

Evolution: Lamarck vs. Darwin

Lamarcks These: *Erworbene Eigenschaften / Eigenschaften in Gebrauch werden vererbt*



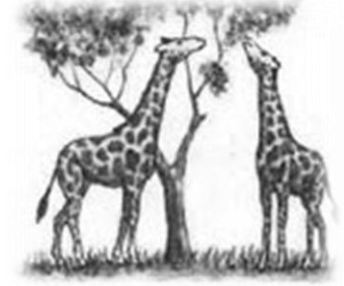
(1744-1829)



Die ersten Giraffen hatten vermutlich **alle kurze Hälse**, die sie langstreckten, um an die Blätter zu kommen

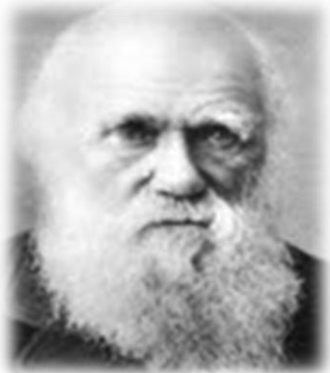


Ihre **Nachkommen** hatten etwas längere Hälse, die sie ebenfalls langstreckten, um an die Blätter zu kommen



Das **stetige Strecken der Hälse** führte zum heutigen Aussehen der Giraffe

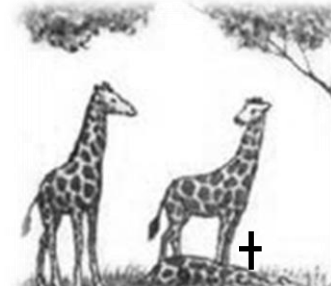
Darwins These: *Veränderungen geschehen zufällig (Mutation), Selektionsprozess lässt die fittesten überleben*



(1809-1882)



Die ersten Giraffen hatten vermutlich **unterschiedlich lange Hälse**



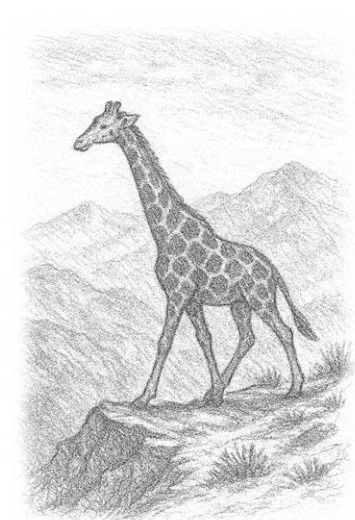
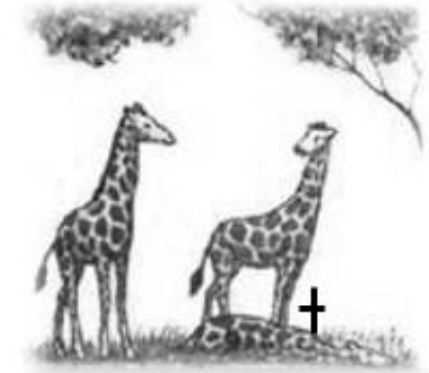
Natürliche Selektion führte aufgrund des Wettbewerbs um Nahrung dazu, dass nur die Giraffen mit längeren Hälsen überlebten und Nachwuchs bekamen



Die heute lebenden Giraffen sind die **Überlebenden dieser „Survival of the Fittest“-Selektion**: Giraffen mit langen Hälsen

Prinzip der natürlichen Selektion

- **Zufällige Generierung einer Lösung**
- **Evaluation** der Lösungen
- **Selektion:**
 - Beibehalten *guter* Lösungen und Verwerfen *schlechter* Lösungen
 - „Gut“ oder „schlecht“ ist *nie* absolut, sondern im *Kontext* der Umgebung zu sehen



Giraffe im Gebirge, wo es keine Bäume gibt?

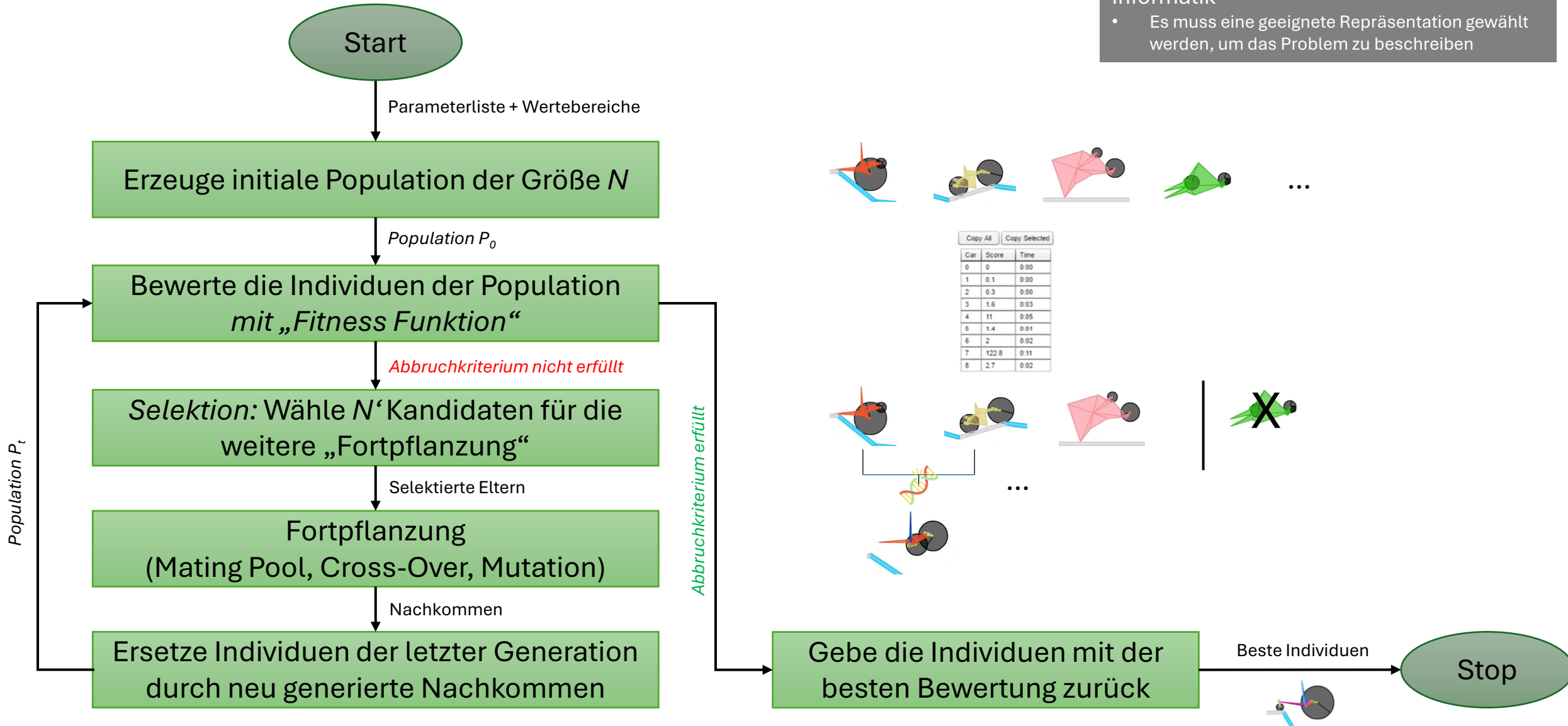
Der grundlegende Algorithmus

Biologie:

- Chromosomen & Gene

Informatik

- Es muss eine geeignete Repräsentation gewählt werden, um das Problem zu beschreiben



Chromosom & Fitness bei BoxCar2D

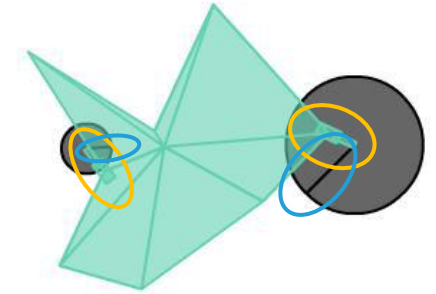
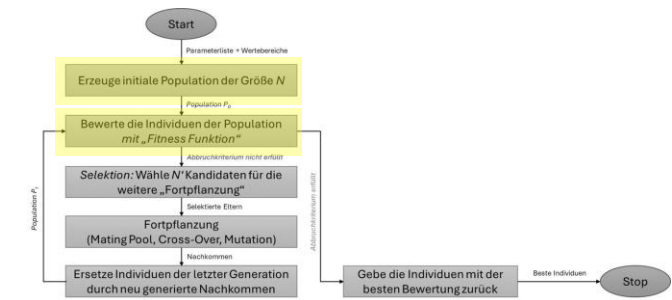


- Jedes Auto besteht aus **8 zufälligen Vektoren** mit **Richtung** und **Länge** ausgehend von Nullpunkt, verbunden durch Dreiecke
- Für **jedes Rad** (# = Parameter) wird zufällig ein Knoten für die Achse gewählt, anschließend wird
 - eine **Ausrichtung** für die Achsaufhängung $[0, 2\pi]$ gesampelt
 - ein **Radius** gesampelt
- Das **Genom** sieht also so aus: $(16 + 3 \cdot 2 = 22 \text{ Variablen / Gene})$

CartAngle ₀	CartMag ₀	CartAngle ₁	CartMag ₁	...	CartAngle ₇	CartMag ₇	WheelVertex ₀	AxleAngle ₀	WheelRadius _{s₀}	WheelVertex ₁	AxleAngle ₁	WheelRadius _{s₁}
------------------------	----------------------	------------------------	----------------------	-----	------------------------	----------------------	--------------------------	------------------------	--------------------------------------	--------------------------	------------------------	--------------------------------------

Fitness Funktion:

- Bewertet, wie gut eine Lösung das optimale Ziel oder die Zielfunktion des Optimierungsproblems erfüllt (hohe Erfüllung = hohe Fitness)
- *Hier*: Zurückgelegte Strecke

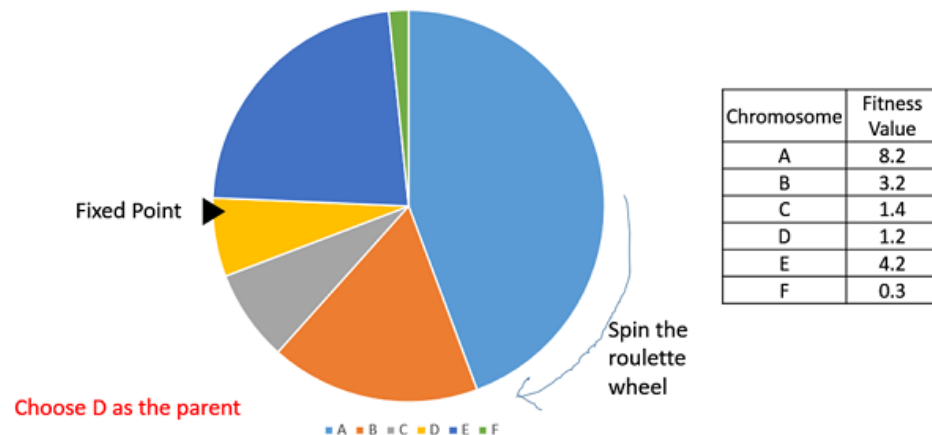


Source: <http://www.boxcar2d.com/about.html>

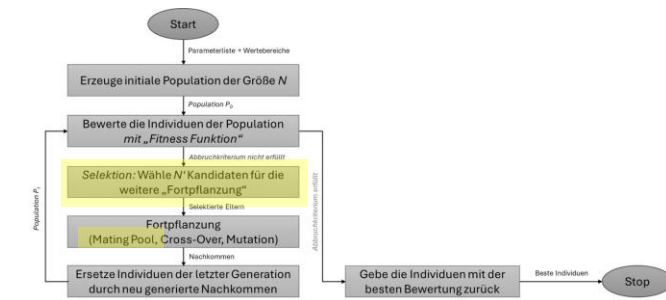
Selektion bei BoxCar2D

Strategie 1: Roulette-Rad

- Wähle Eltern basierend auf ihrem *normalisierten Fitness Score*
- Normalisiert bedeutet: Fitness Score geteilt durch die Summe aller Fitness Scores der aktuellen Generation
- *Hier*: $N/2$ mal Sampeln ohne Zurücklegen + Verbesserungen



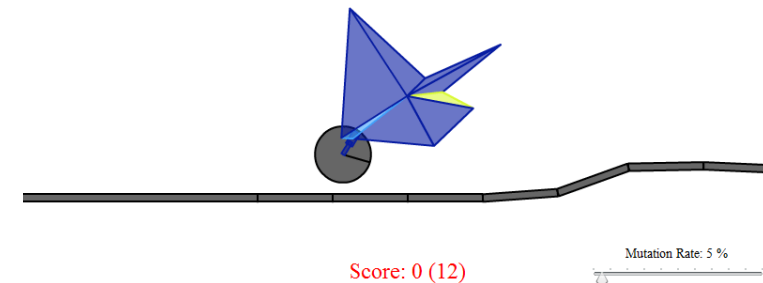
Source: https://www.tutorialspoint.com/genetic_algorithms/genetic_algorithms_parent_selection.htm



48 fps average
Physics step: 5 ms (186 fps)
30 MB used
#0 : 3.9
#1 : 0.2
#2 : 6
#3 : 1

Generation 1

$N=4$



score	probability	roulette wheel piece
3.9	35.1%	0 to 35.1
0.2	1.8%	35.2 to 36.9
6.0	54.1%	37 to 91
1.0	9.0%	91 to 100

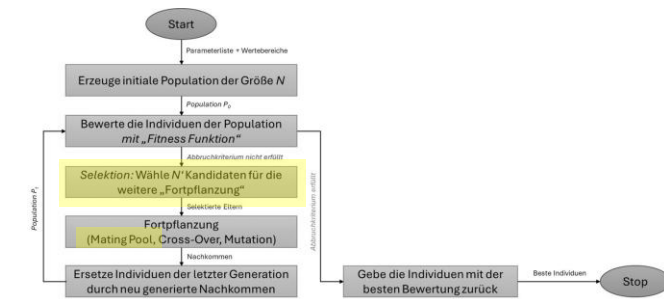
Source: <http://www.boxcar2d.com/about.html>

Selektion bei BoxCar2D

Strategie 2: Tournament Selection

- Bessere Kontrolle, vermindert den Selektionsdruck (wenige dominante Gene entscheidend, werden stetig rekombiniert, Diversität der Population geht früh verloren)
- *Pseudocode:*

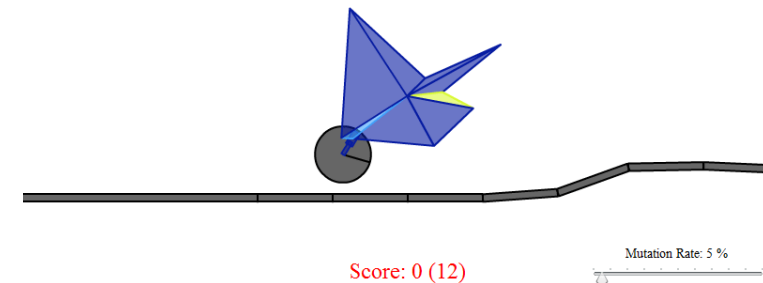
```
for each car A
  *pick a random car B (excluding A)
  *highest score of A and B wins tournament
  *put winner in mating pool
randomly pick pairs from mating pool for crossover
```



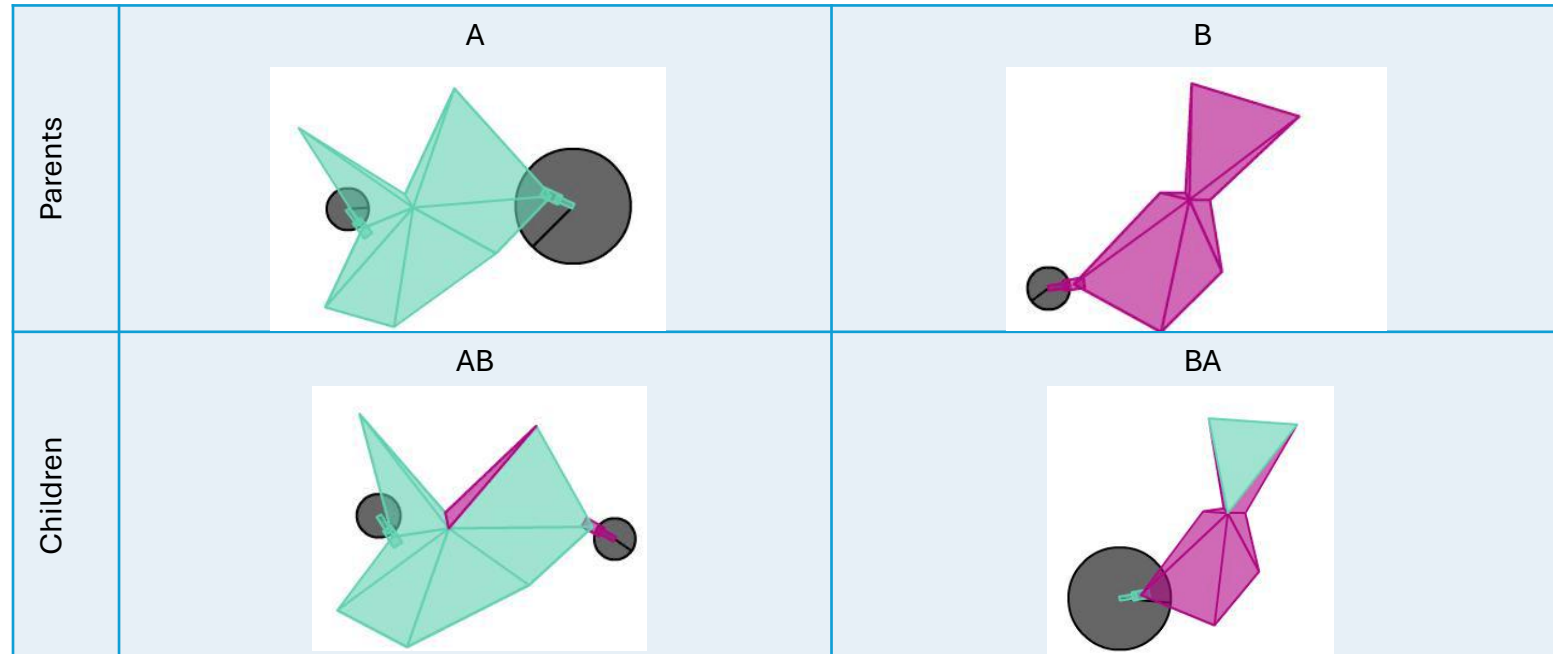
48 fps average
Physics step: 5 ms (186 fps)
30 MB used
#0 : 3.9
#1 : 0.2
#2 : 6
#3 : 1

Generation 1

$N=4$

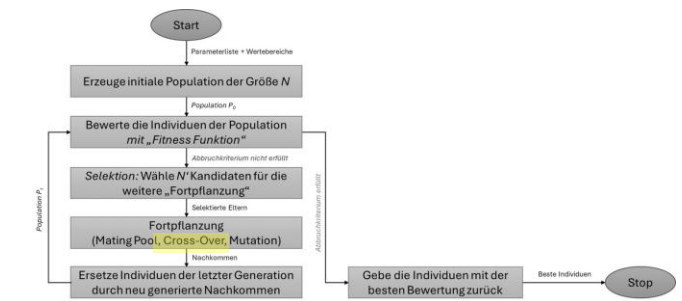


Fortpflanzung bei BoxCar2D: Crossover



Source: <http://www.boxcar2d.com/about.html>

Car	Angle0	Mag0	Angle1	Mag1	...	...	...	...	...	...	...	...	...	...	...	...	WheelV rtex0	AxleAngl e0	WheelRa dius0	WheelV rtex1	AxleAngl e1	WheelRa dius1
A	0.769	2.614	0.584	0.319	0.278	2.883	0.666	1.13	0.305	2.752	0.376	2.507	0.814	1.963	0.392	2.872	3	5.284	0.434	7	2.625	1.191
B	0.535	2.682	0.732	2.256	0.422	0.149	0.676	0.578	0.709	2.774	0.592	2.623	0.519	1.531	0.924	0.404	-1	0.704	0.122	4	0.167	0.409
AB	0.535	2.682	0.584	0.319	0.278	2.883	0.666	1.13	0.305	2.752	0.376	2.507	0.814	1.963	0.392	2.872	3	5.284	0.434	7	2.625	0.409
BA	0.769	2.614	0.732	2.256	0.422	0.149	0.676	0.578	0.709	2.774	0.592	2.623	0.519	1.531	0.924	0.404	-1	0.704	0.122	4	0.167	1.191

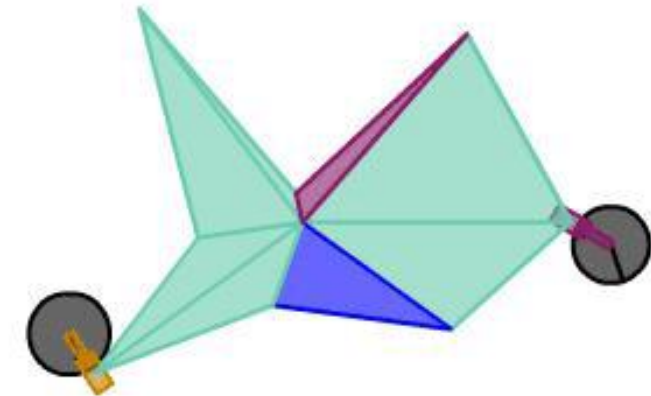
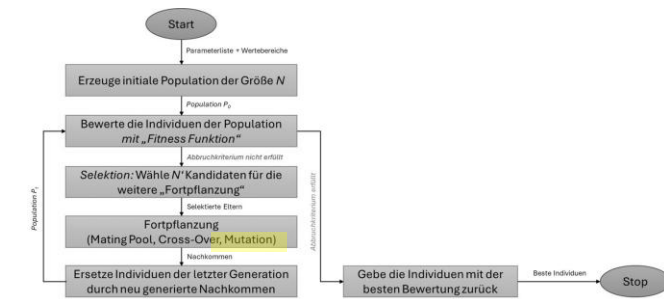


“Two point Crossover”:

- Zufällige Wahl zweier Punkte entlang des Chromosoms, tauschen aller Einträge dazwischen
- Hier: Dritte und vorletzte Position
- Führt dazu, dass BA das große Rad von A erhält und BA das Kleine
- Beachte: Zwei Elternteile mit hoher Fitness können, müssen aber keine Nachkommen mit hoher fitness erzeugen

Fortpflanzung bei BoxCar2D: Mutation

- Jede Eigenschaft / Variable des Fahrzeugs wird *mit gewisser, einstellbarer Wahrscheinlichkeit verändert*
- (100% = komplett zufällige Nachkommen ohne Vererbung)
- Wenn eine Variable mutiert, wird ein zufälliger Wert aus dem *zulässigen Wertebereich* gesampelt
- Dies wird durch eine zufällig gesampelte neue Farbe visualisiert



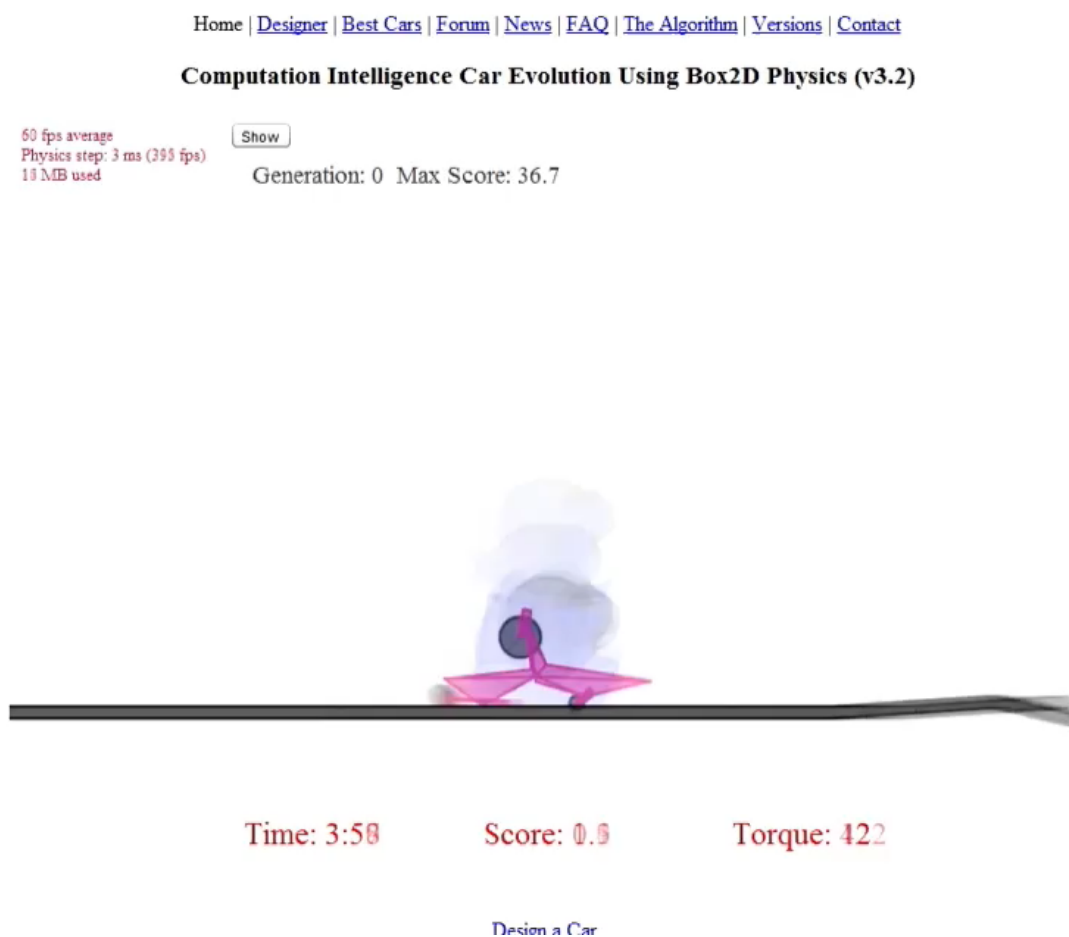
Beispiel mit zwei mutierten Variablen/Gene (nach Crossover):

- Eine Vektorlänge (blau)
- Eine Position eines der Räder (orange)

Car	Angle0	Mag0	...	...	...	...	...	...	...	...	Angle5	Mag5	...	...	...	...	WheelV ertex0	AxleAng le0	WheelR adius0	WheelV ertex1	AxleAng le1	WheelR adius1
AB	0.535	2.682	0.584	0.319	0.278	2.883	0.666	1.13	0.305	2.752	0.376	2.507	0.814	1.963	0.392	2.872	3	5.284	0.434	7	2.625	0.409
AB _m	0.535	2.682	0.584	0.319	0.278	2.883	0.666	1.13	0.305	2.752	0.376	0.940	0.814	1.963	0.392	2.872	4	5.284	0.434	7	2.625	0.409

Source: <http://www.boxcar2d.com/about.html>

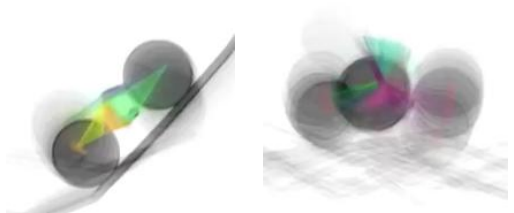
BoxCar 2D



Erste Iterationen



2-Rad setzt sich durch



40+: optimiertes 2-Rad + 3-Rad

Finde das Maximum der Funktion $f(x_1, x_2) = \cos(0.3 \cdot x_1 \cdot x_2) + 1.5$

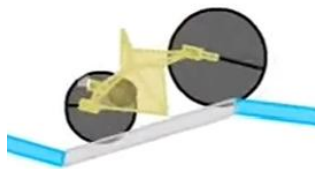
Wie kann eine **geeignete Repräsentation** für dieses Problem aussehen?

Ansatz:

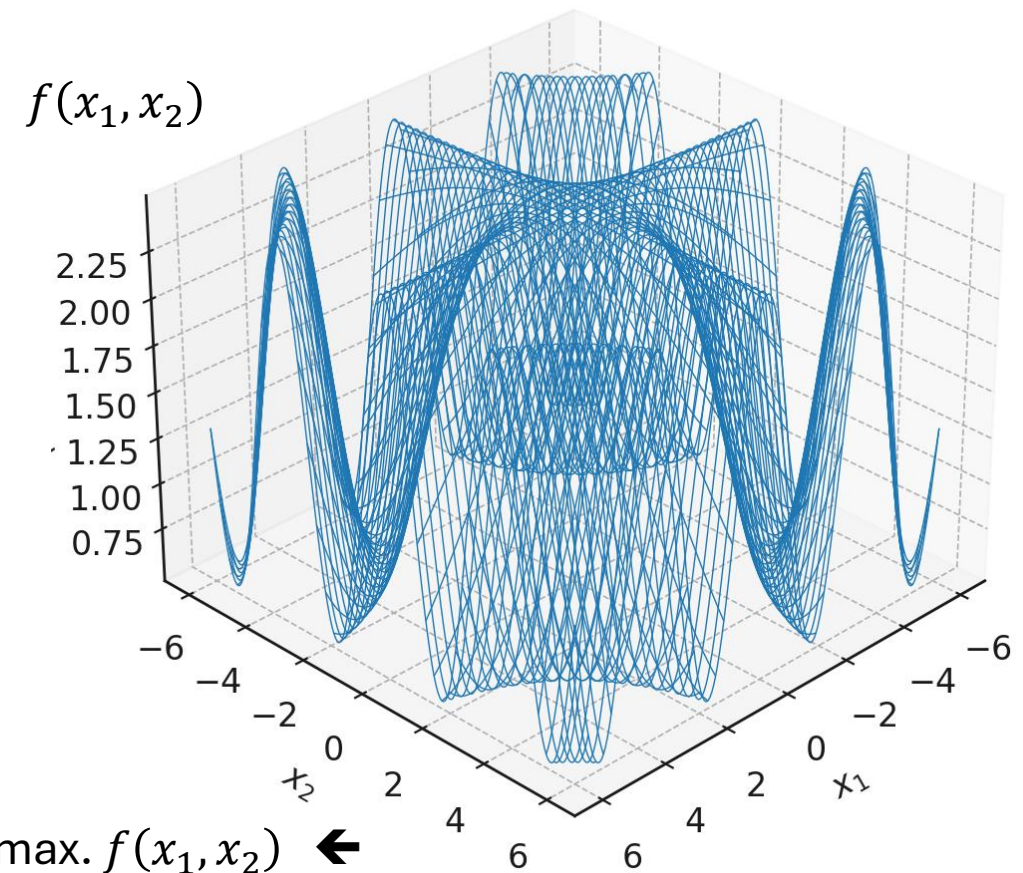
- Chromosom repräsentiert durch 2D Punkt
- Population/Individuen gespeichert in 2D Array:

x_1	x_2	
[2.77816827	3.246249]	# "Individuum1"
[2.36282656	-5.24630732]	# "Individuum2"
[-1.22549777	-5.53211536]	# "Individuum3"
[5.17849005	5.41245122]	
...		

- Selektion? Wähle Kandidat mit max. Fitness, d.h.



➔ max. zurückgelegte Strecke



Lösungssuche: Cross-Over

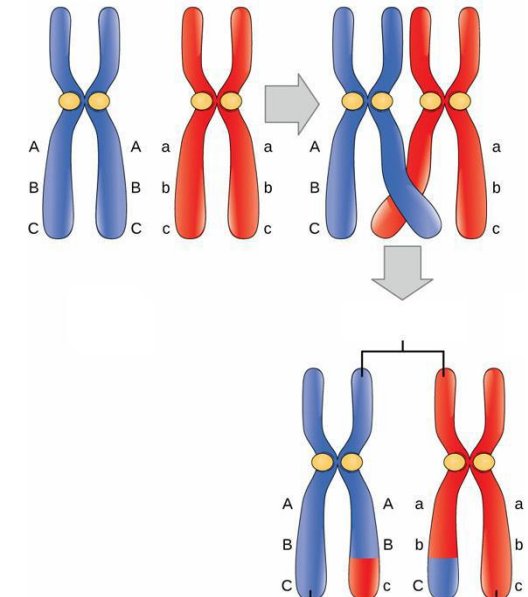
Biologie: Austausch von genetischem Material zwischen Vater und Mutter

Informatik: Austausch von Informationen, die das *gleiche Feature* codieren

Homologie: Gleiche Genorte!

```
      x1          x2  
[[ 2.77816827 3.246249 ] # "Individuum1"  
[ 2.36282656 -5.24630732] # "Individuum2"  
[-1.22549777 -5.53211536] # "Individuum3"  
[ 5.17849005 5.41245122]  
...
```

```
      x1          x2  
[[ 2.36282656 3.246249 ] # "Individuum1"  
[ 2.77816827 -5.24630732] # "Individuum2"  
[-1.22549777 -5.53211536] # "Individuum3"  
[ 5.17849005 5.41245122]  
...
```



Lösungssuche: Mutation

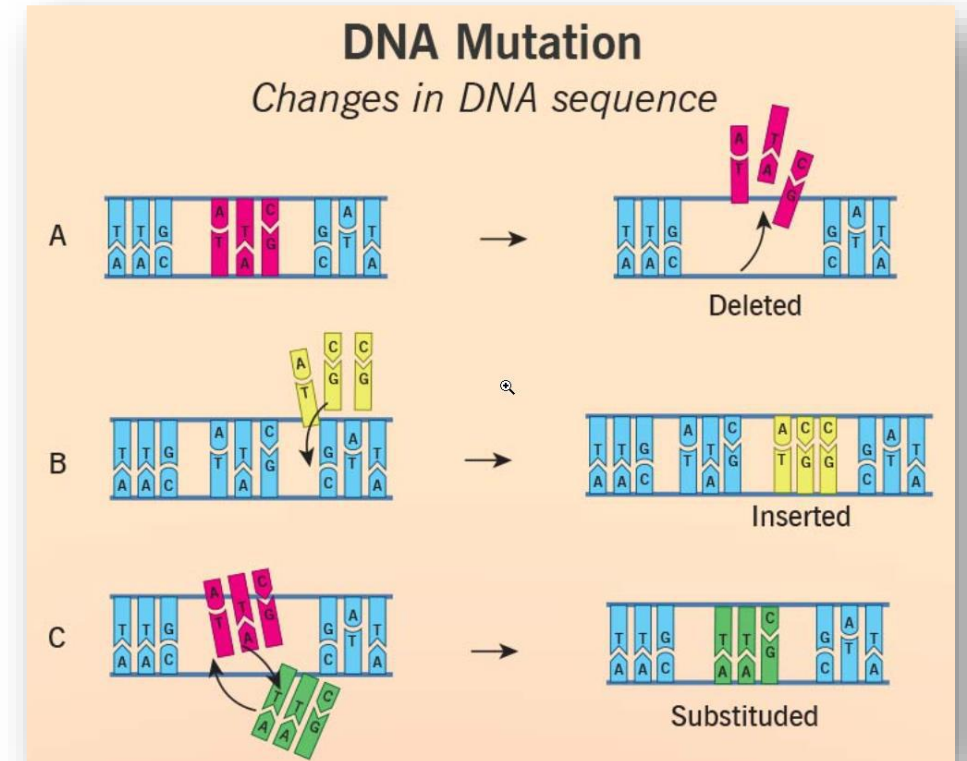
Biologie: Veränderung eines Genortes

Informatik: Veränderung einer Information

```
      x1          x2  
[[ 2.36282656 3.246249 ] # "Individuum1"  
[ 2.77816827 -5.24630732] # "Individuum2"  
[-1.22549777 -5.53211536] # "Individuum3"  
[ 5.17849005 5.41245122]  
...
```

```
      x1          x2  
[[ 2.36282656 3.246249 ] # "Individuum1"  
[ 0.07456000 -5.24630732] # "Individuum2"  
[-1.22549777 -5.53211536] # "Individuum3"  
[ 5.17849005 5.41245122]  
...
```

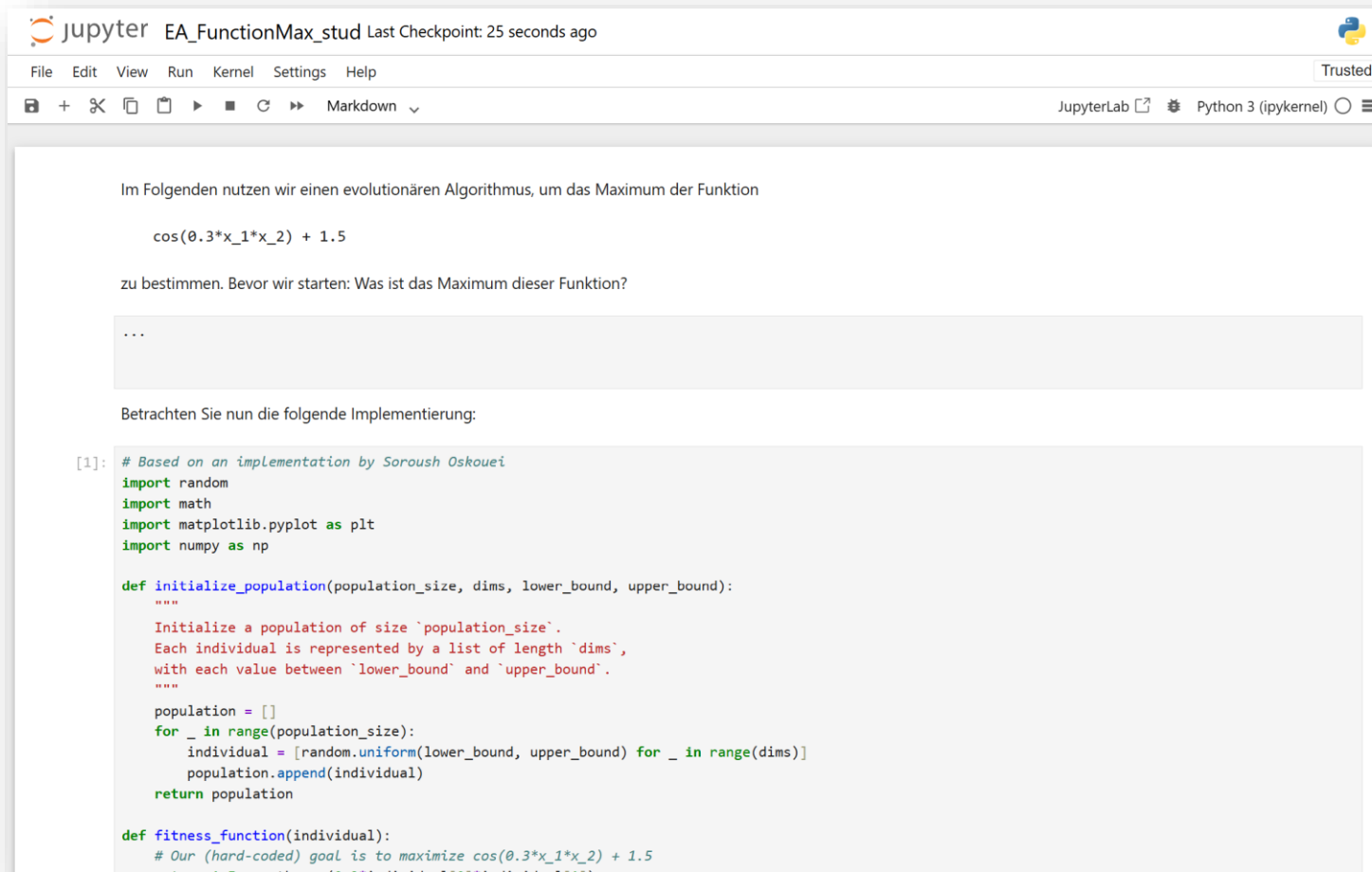
Deletion, Insertion, Substitution:



Source: <https://studyx.ai/questions/4lgnl26/dna-mutationchanges-in-dna-sequence>
the-image-contains-three-types-of-dna-mutations

Wir nutzen nur Substitution
(analog zum BoxCar2D Beispiel)

Übungsaufgabe: Maximum einer Funktion



```
Im Folgenden nutzen wir einen evolutionären Algorithmus, um das Maximum der Funktion

cos(0.3*x_1*x_2) + 1.5

zu bestimmen. Bevor wir starten: Was ist das Maximum dieser Funktion?

...

Betrachten Sie nun die folgende Implementierung:
```

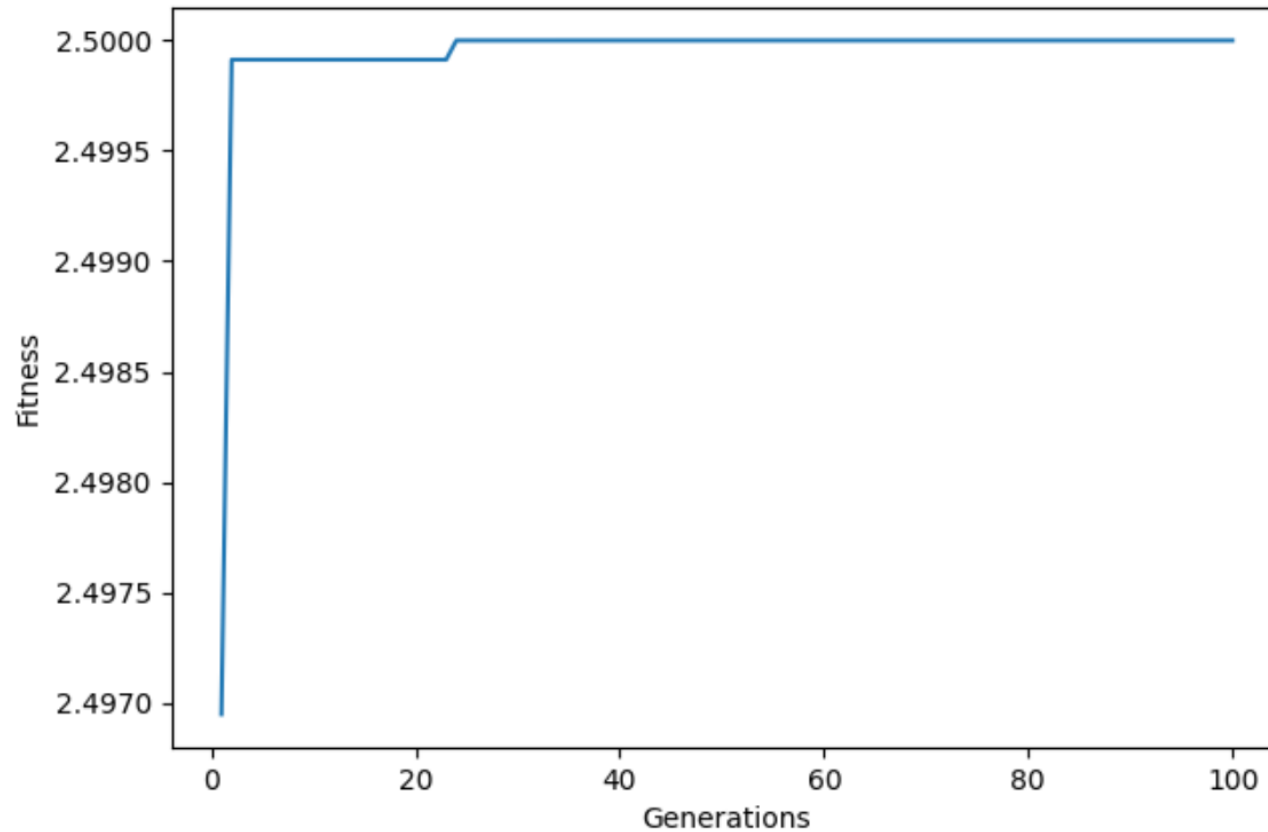
```
[1]: # Based on an implementation by Soroush Oskoueï
import random
import math
import matplotlib.pyplot as plt
import numpy as np

def initialize_population(population_size, dims, lower_bound, upper_bound):
    """
    Initialize a population of size `population_size`.
    Each individual is represented by a list of length `dims`,
    with each value between `lower_bound` and `upper_bound`.
    """
    population = []
    for _ in range(population_size):
        individual = [random.uniform(lower_bound, upper_bound) for _ in range(dims)]
        population.append(individual)
    return population

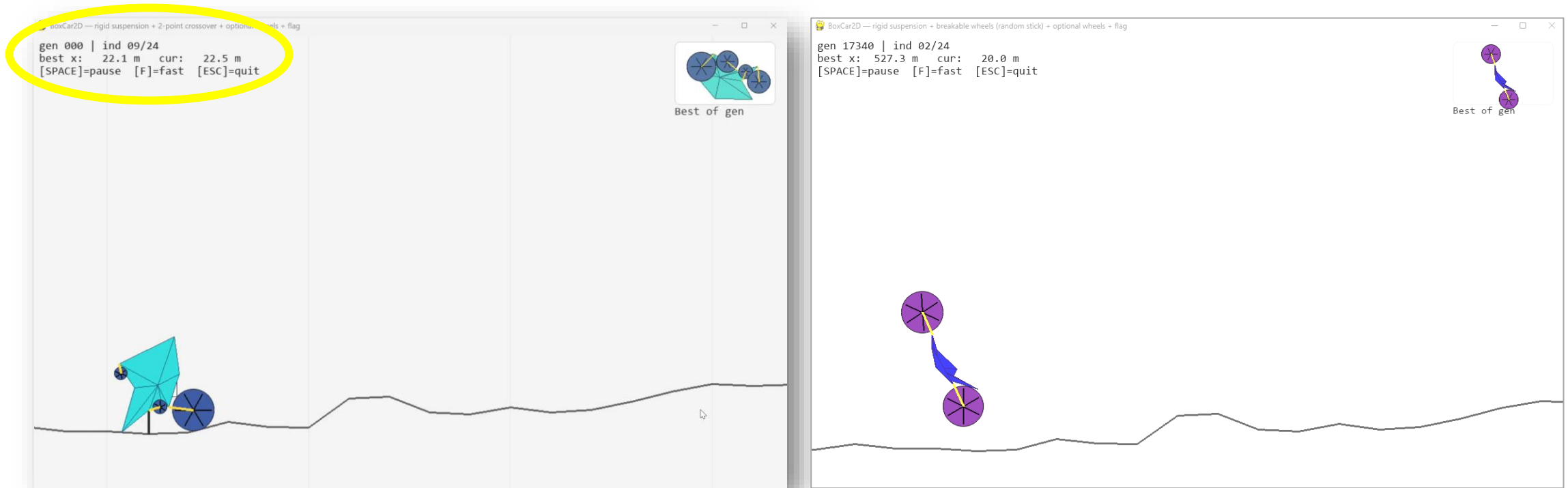
def fitness_function(individual):
    # Our (hard-coded) goal is to maximize cos(0.3*x_1*x_2) + 1.5
    return 1.5 + math.cos(0.3*individual[0]*individual[1])
```

- Finden des Maximums der Funktion $f(x_1, x_2) = \cos(0.3 \cdot x_1 \cdot x_2) + 1.5$
- *Dazu:* Implementierung von Crossover und Mutation

Übungsaufgabe: Maximum einer Funktion



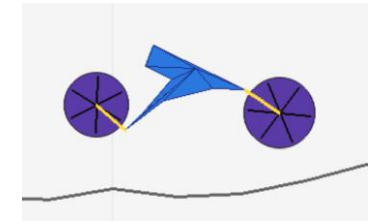
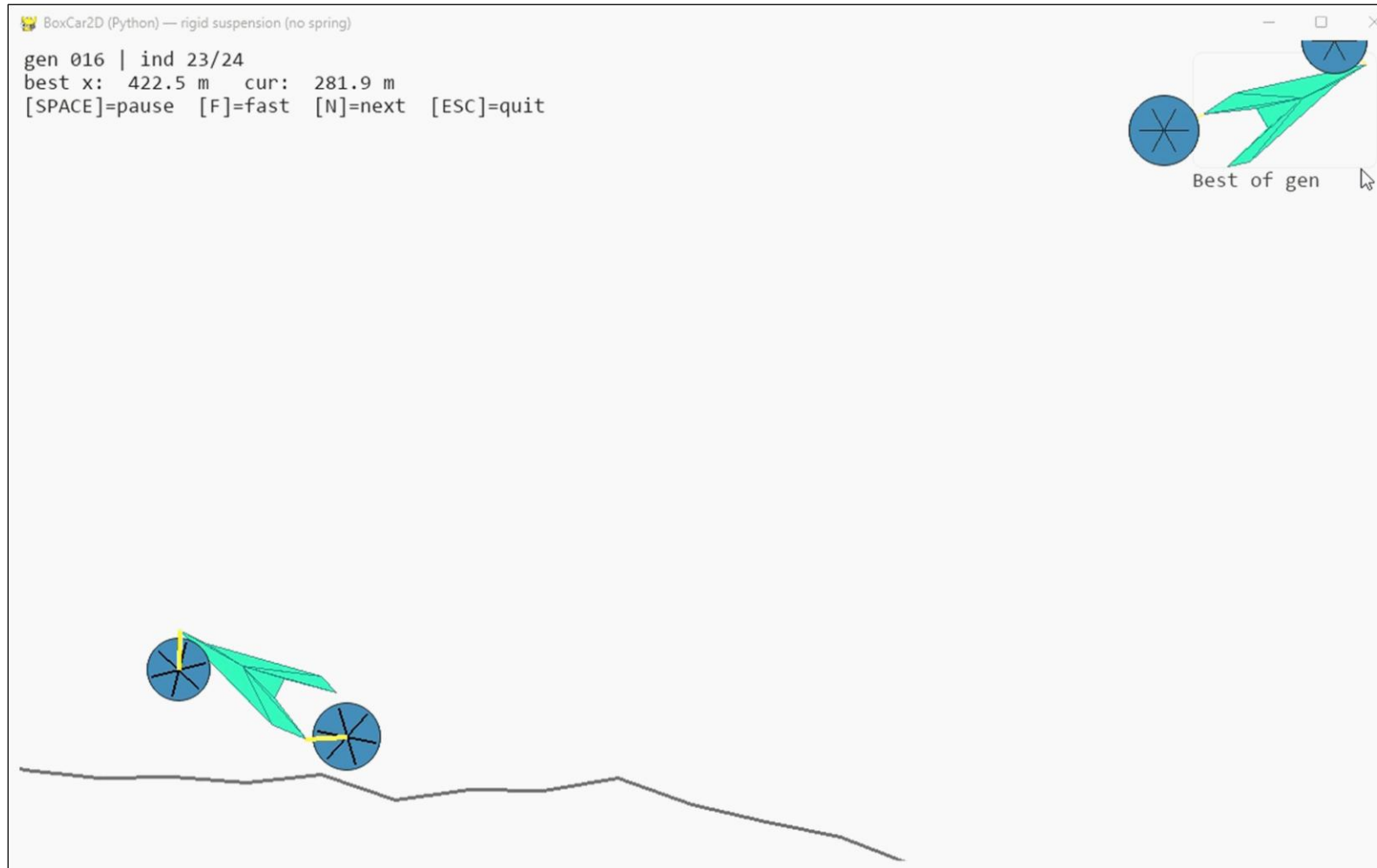
Quick & Dirty Vibe-Coding Projekt (ChatGPT 5)



Hinweise:

- ich erlaube hier minimal 2 und maximal bis zu vier Räder, letztere 2 haben im Genom einen bool Wert für Existenz
- Farbgebung abgeleitet aus Genom, ähnliches Genom → ähnliche Farbe

Quick & Dirty Vibe-Coding Projekt (ChatGPT 5)



```
Generation 0: best=232.30 avg=52.07
Generation 1: best=232.30 avg=55.93
Generation 2: best=232.30 avg=65.55
Generation 3: best=232.30 avg=91.69
Generation 4: best=232.30 avg=63.29
Generation 5: best=312.52 avg=82.93
Generation 6: best=312.52 avg=93.97
Generation 7: best=312.52 avg=90.15
Generation 8: best=348.63 avg=85.53
Generation 9: best=384.94 avg=143.86
Generation 10: best=401.33 avg=137.12
Generation 11: best=401.33 avg=164.41
Generation 12: best=401.33 avg=188.16
Generation 13: best=408.29 avg=160.58
Generation 14: best=408.29 avg=230.79
Generation 15: best=422.53 avg=224.89
Generation 16: best=422.53 avg=265.08
Generation 17: best=422.53 avg=255.52
```

⋮

```
Generation 258: best=502.13 avg=367.47
Generation 259: best=502.13 avg=271.55
Generation 260: best=502.13 avg=269.70
Generation 261: best=502.13 avg=285.52
Generation 262: best=502.13 avg=273.00
Generation 263: best=502.13 avg=278.59
```

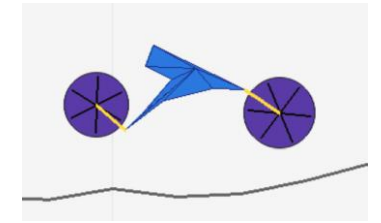
Frage...

Kann der „best“ Wert bei Verwendung von Elitismus (d.h., die besten Exemplare nehmen wir mit in die nächste Generation) auch wieder schlechter werden?

Antwort: Es kommt darauf an!

Verschlechterungen sind z.B. möglich wenn...

- das **Environment** nichtdeterministische Anteile enthält (z.B. Strecke ändert sich, bei Radbruch wird aus einer Verteilung gesampelt etc.)
- Irgendwo ein **Optimierer** mit Zufallsanteilen benutzt wird o.ä.



```
Generation 0: best=232.30 avg=52.07
Generation 1: best=232.30 avg=55.93
Generation 2: best=232.30 avg=65.55
Generation 3: best=232.30 avg=91.69
Generation 4: best=232.30 avg=63.29
Generation 5: best=312.52 avg=82.93
Generation 6: best=312.52 avg=93.97
Generation 7: best=312.52 avg=90.15
Generation 8: best=348.63 avg=85.53
Generation 9: best=384.94 avg=143.86
Generation 10: best=401.33 avg=137.12
Generation 11: best=401.33 avg=164.41
Generation 12: best=401.33 avg=188.16
Generation 13: best=408.29 avg=160.58
Generation 14: best=408.29 avg=230.79
Generation 15: best=422.53 avg=224.89
Generation 16: best=422.53 avg=265.08
Generation 17: best=422.53 avg=255.52
```

⋮

```
Generation 258: best=502.13 avg=367.47
Generation 259: best=502.13 avg=271.55
Generation 260: best=502.13 avg=269.70
Generation 261: best=502.13 avg=285.52
Generation 262: best=502.13 avg=273.00
Generation 263: best=502.13 avg=278.59
```

BoxCar2D Reimplementierung braucht...

- **Python 3.10** (aktuellere Versionen machen Probleme)
- Packages **box2d-py** & **pygame**
- Setup Hinweise mit Hilfe von **Anaconda** auf den folgenden Folien:

BoxCar2D Setup – Miniconda (Kommandozeile)

Ziel: Lokale Umgebung für BoxCar2D mit Python 3.10

- **Miniconda installieren:**
 - Download: <https://docs.conda.io/en/latest/miniconda.html>
- **Neue Umgebung anlegen:**
 - `conda create -n boxcar python=3.10 -y`
 - `conda activate boxcar`
- **Pakete installieren** (Channel conda-forge):
 - `conda install -c conda-forge box2d-py pygame -y`
- **Code ausführen:**
 - `python boxcar2d_evo.py`
- **Optional: GUI hinzufügen:**
 - `conda install anaconda-navigator -y`

BoxCar2D Setup – Anaconda Navigator (GUI)

Ziel: Umgebungserstellung über die grafische Oberfläche.

- **Navigator starten:**
 - Anaconda Navigator öffnen (Startmenü / Spotlight)
- **Neue Environment erstellen:**
 - Environments → Create
 - Name: boxcar · Python-Version: 3.10 · Create
- **Pakete installieren:**
 - Environment 'boxcar' auswählen
 - Channel „conda-forge“ dazufügen
 - Filter: Not Installed
 - Suchen & installieren: box2d-py, pygame
- **Code starten:**
 - Home → Terminal / VS Code öffnen
 - `python boxcar2d_evo.py`

DEAP Framework

- Evolutionäre Algorithmen Framework: **D**istributed **E**volutionary **A**lgorithms in **P**ython
- Module für die Erstellung von Individuen, Operatoren und Repräsentationen
 - Mit der Klasse `creator` können zur Laufzeit neue Klassen generiert werden

```
from deap import base, creator

creator.create("FitnessMin", base.Fitness, weights=(-1.0,)) # "-1" means minimization ("+1"=max)
creator.create("Individual", list, fitness=creator.FitnessMin) # individual has list of genes + fitness
```

- Der Container `toolbox` implementiert genetische Operatoren

```
import random
from deap import tools

GENOME_SIZE = 10 # number of genes of each individuum
# initiate toolbox & register (i.e., define) properties of the evolutionary algorithm & representation:
toolbox = base.Toolbox()
toolbox.register("attribute", random.random) # register function that generates 1 random number in [0,1]
# Define how an individual of type Individual is generated: here by n times calling function „attribute“
toolbox.register("individual", tools.initRepeat, creator.Individual, toolbox.attribute, n=GENOME_SIZE)
toolbox.register("population", tools.initRepeat, list, toolbox.individual) # create popul. of individuals
```

DEAP Framework

toolbox

- |
 - |— "attribute"
 - | └─ erzeugt ein **Gen** (z.B. Zufallswert 0..1 mit random.random oder auch ein Tupel)
- |
 - |— "individual"
 - | └─ erzeugt eine Liste von Genen
 - | └─ **Anzahl Gene** = GENOME_SIZE
 - | └─ Typ = creator.Individual
- |
 - |— "population"
 - | └─ erzeugt eine Liste von Individuen
 - | └─ **Anzahl Individuen** = Populationsgröße (Aufruf mit toolbox.population(n=300))

DEAP Framework

- Genetische Operatoren:

```
def evaluate(individual):  
    return sum(individual), # DEAP expects fitness values as tuples, achieved by ","  
  
# two point crossover: exchange genes between both points  
toolbox.register("mate", tools.cxTwoPoint)  
  
# Gaussian mutation (additive with Gaussian params mu & sigma), mutation probability for each  
# gene is 10%  
toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=1, indpb=0.1)  
  
# Randomly select tournsize individuals from population, the one with the highest fitness  
# score wins, alternative: toolbox.register("select", tools.selBest) (elitism)  
toolbox.register("select", tools.selTournament, tournsize=3)  
  
# set link to the above function  
toolbox.register("evaluate", evaluate)
```

DEAP Framework: Function maximization

```
# -----
# DEAP: classes & toolbox
# -----

# Maximize (weight +1.0)
creator.create("FitnessMax", base.Fitness, weights=(1.0,))

# An Individual is just a python list with an attached Fitness
creator.create("Individual", list, fitness=creator.FitnessMax)

toolbox = base.Toolbox()

# Attribute: one real value in [LOWER_BOUND, UPPER_BOUND]
toolbox.register("attr_float", random.uniform, LOWER_BOUND,
UPPER_BOUND)

# Individual: repeat attribute DIMS times
toolbox.register("individual", tools.initRepeat,
creator.Individual, toolbox.attr_float, n=DIMS)

# Population: list of individuals
```

```
toolbox.register("population", tools.initRepeat, list,
toolbox.individual)

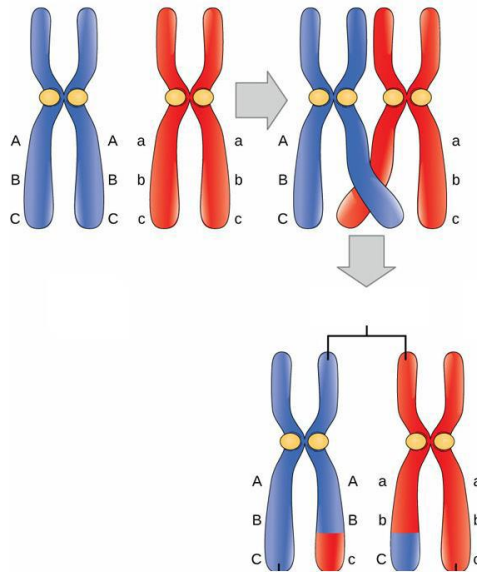
# Register functions
toolbox.register("evaluate", fitness_func)
toolbox.register("mate", crossover_swap)
toolbox.register("mutate", mutation, low=LOWER_BOUND,
up=UPPER_BOUND, indpb=MUT_INDPB)

# Roulette (fitness-proportionate) selection
toolbox.register("select", tools.selRoulette)

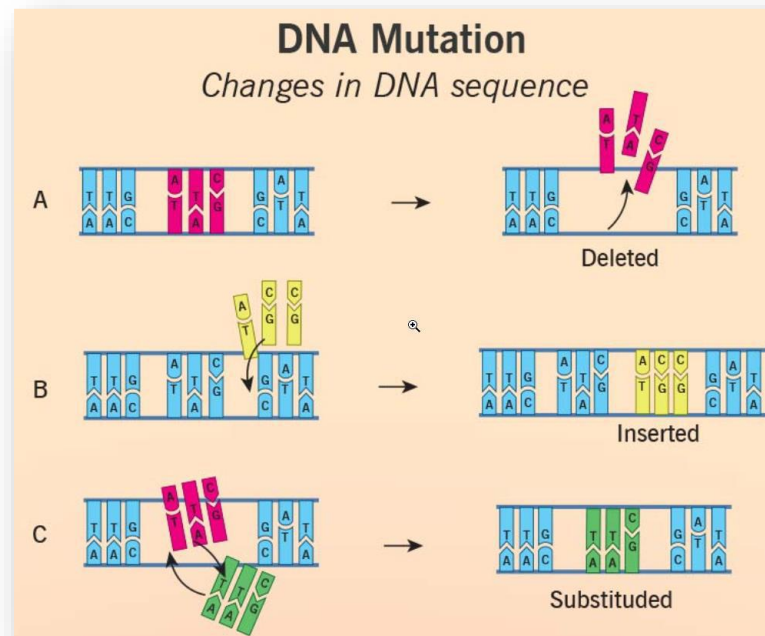
[...]
def fitness_func(ind):
    return 1.5 + math.cos(0.3 * ind[0] * ind[1])
```

Genetische Operatoren (Wdh)

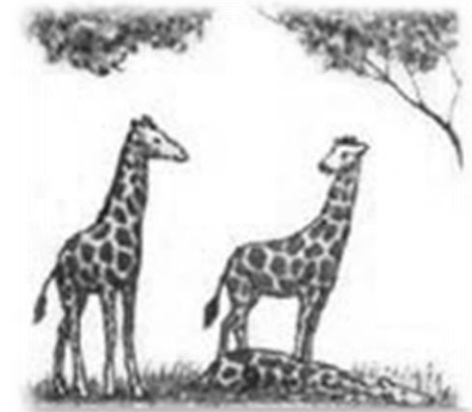
Als **Genetische Operatoren** bezeichnet man die Mechanismen, die in evolutionären Algorithmen die Lösungskandidaten verändern, z.B. **Crossover**, **Mutation**, **Selektion**



Austausch von Teillösungen



Veränderung von Teillösungen



Auswahl der besten Lösungskand.

Genetische Operatoren in DEAP: Crossover

Repräsentations- form

DEAP Implementierung

Binär

1-Punkt Crossover



N-Punkt Crossover



```
deap.tools.cxOnePoint(ind1, ind2)
deap.tools.cxUniform(ind1, ind2, i
ndpb)
```

Integer

Wie binär

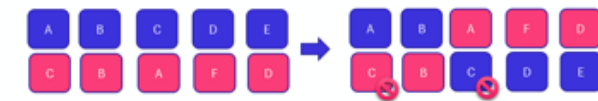
Floating Point

Wie binär & arithmetische Rekombination: z. B. Bilden eines Mittwertes



Permutation

Eine Rekombination wie bei den vorherigen Ansätzen ist nicht praktikabel / resultiert in falschen Lösungen



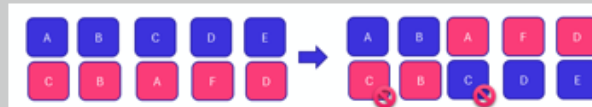
Genetische Operatoren in DEAP: Crossover

Repräsentations- form

DEAP Implementierung

Permutation

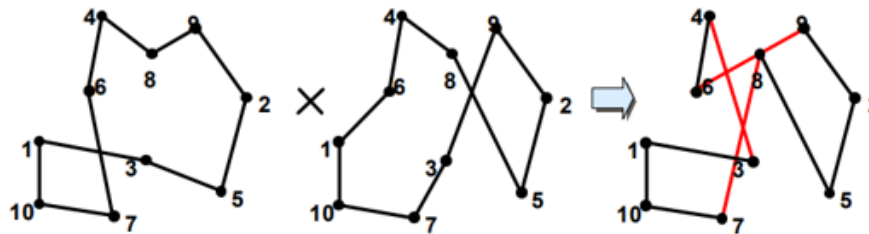
Eine Rekombination wie bei den vorherigen Ansätzen ist nicht praktikabel / resultiert in falschen Lösungen



Eine (von vielen) Lösungen:

z. B. partially mapped crossover (PMX): die rot markierten Kanten stammen von keinem der beiden Eltern

`deap.tools.cxPartiallyMatched(ind1, ind2)`



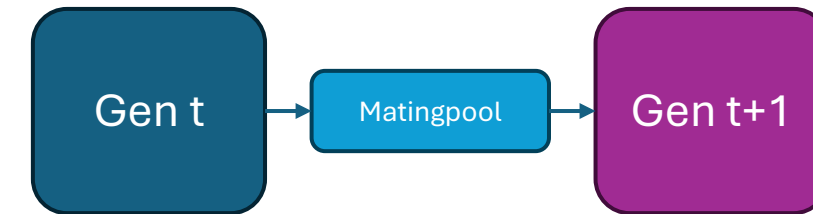
Genetische Operatoren in DEAP: Mutation

Repräsentations- form		DEAP Implementierung
Binär	Jedes Bit wird mit einer bestimmten Wahrscheinlichkeit (<u>indpb</u>) geflippt	<code>deap.tools.mutFlipBit(individual, <u>indpb</u>)</code>
Integer	<i>Random</i> : Zufälliges Setzen eines Wertes <i>Creep</i> : Zufällige kleine Veränderung	
Floating Point	Zufälliges Setzen eines Wertes in bestimmten Grenzen Zufällige Veränderung gemäß Verteilung (z.B. <u>Gauss</u> - macht große Veränderungen unwahrscheinlicher)	<code>deap.tools.mutUniformInt(individual, low, up, <u>indpb</u>)</code> <code>deap.tools.mutGaussian(individual, mu, sigma, <u>indpb</u>)</code>
Permutation	Swap (ABCDE -> A <u>DC</u> B <u>E</u>) : B und D vertauscht Insert(ABCDE -> A <u>C</u> BDE) : C in Richtung A verschoben <u>Scramble</u> (ABCDE -> CABDE) mischen eines Abschnitts Inversion(ABCDE -> CBADE) Inversion eines Abschnitts	<code>deap.tools.mutShuffleIndexes(individual, <u>indpb</u>)</code>

Genetische Operatoren in DEAP: Selektion

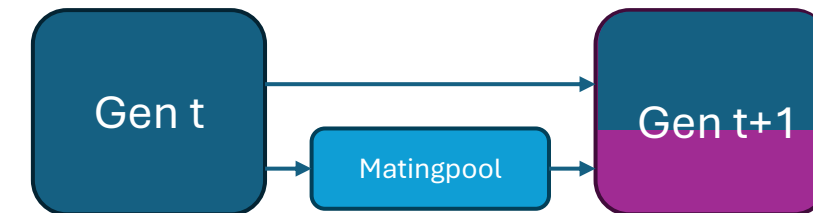
- Via **Generationenmodell:**

- Jede Generation wird eine *komplett neue Population* erzeugt (meist gleich groß wie die alte), die die alte ersetzt
- Optional: **Elitenismus**: Übernahme die besten n Individuen unverändert
- *Eigenschaft*: Große Sprünge zwischen den Generationen (Exploration > Exploitation)
- *In DEAP*: `algorithms.eaSimple` oder `eaGenerational` (erlaubt eigene Selektionsstrategien)



- Via **Steady-State-Modell:**

- Pro Iterationsschritt werden **nur wenige Individuen** selektiert, gekreuzt, mutiert
- Diese ersetzen direkt die **schlechtesten Individuen** in der Population (oder zufällige), die meisten alten Individuen bleiben erhalten
- *Eigenschaft*: Generationen überlappen, Individuen haben unterschiedliches Alter, langsamere, stetigere Fortschritte (Exploitation > Exploration)
- *In DEAP*: `algorithms.eaMuPlusLambda` oder `eaMuCommaLambda`



Genetische Operatoren in DEAP: Selektion

Bestenliste	Wähle die k besten Individuen.	<code>deap.tools.selBest(individuals, k, fit_attr='fitness')</code>
Wettkampf	Wähle das beste Individuum aus einer Gruppe von k zufällig ausgewählten Individuen.	<code>deap.tools.selTournament(individuals, k, tournsize, fit_attr='fitness')</code>

Evolutionary Computing

Teil 3: Anwendungen



Anwendungsgebiete 1/2



- **Parameteroptimierung**

- Finden von Parametern, so dass eine reell-wertige Bewertungsfunktion ein (idealerweise globales) Optimum annimmt



- **Packprobleme: Knapsack**

- z.B. Füllen eines Rucksacks mit begrenzter Kapazität mit maximalem Wert
- z.B. Packen möglichst weniger Kisten bei gegebenen Gütern



- **Wegeprobleme**

- z.B. Traveling Salesman Problem (→ Praktische Anwendung: Effizientes Bohren von Platinen)
- z.B. Bestimmen der Reihenfolge anzufahrender Ziele
- z.B. Routenoptimierung, Verlegen von Leiterbahnen auf Platinen



- **Anordnungsprobleme**

- z.B. Positionierung von Verteilerknoten in einem Telefonnetz

Anwendungsgebiete 2/2



- **Planungsprobleme**

- z.B. Scheduling/Ablaufplanung, Arbeitspläne, Operationsfolgen
- z.B. Compileroptimierungen: Umordnung von Befehlen



- **Strategieprobleme**

- z.B. Gefangenendilemma / Spieltheorie
- Verhaltensmodellierung von Akteuren im Wirtschaftsleben



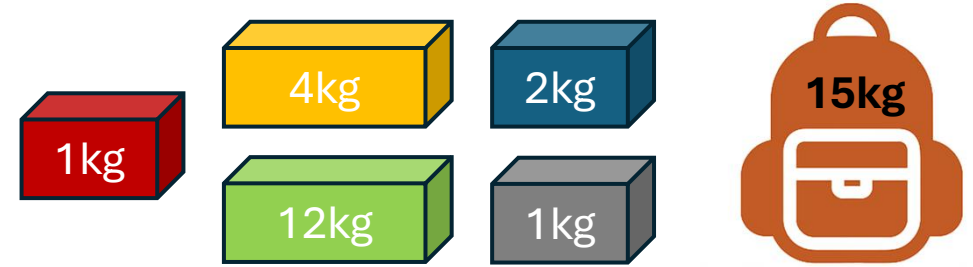
- **Biologische Modellierung**

- z.B. beschreibende Regeln zum Spinnennetzbau
- EA modelliert Parameter, Übereinstimmung mit Natur? Falls ja: Gutes Modell

	Clyde schweigt	Clyde verrät Bonnie
Bonnie schweigt	ein Jahr Freiheitsstrafe 	lebenslänglich
Bonnie verrät Clyde	Freilassung 	zehn Jahre Freiheitsstrafe

Quelle: <https://www.iconomix.ch/de/module/a054/a006/fachtext/>

Bsp: Packproblem / Knapsack



- Kombinatorisches Optimierungsproblem
- **Setting:**
 - **Gegeben:** Set von n **Items**; mit jedem Item sind bestimmte **Kosten** c und ein bestimmter **Wert** w verbunden.
 - **Gesucht:** Minimiere $\sum c$ bei gleichzeitiger Maximierung von $\sum w$

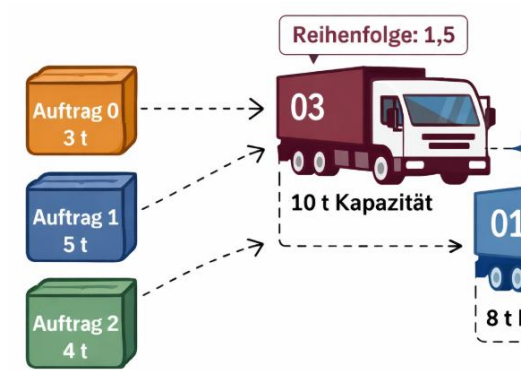
Item	Rote Schachtel	Graue Schachtel	Blaue Schachtel	Gelbe Schachtel	Grüne Schachtel
Wert	2	1	1	8	5
Kosten [kg]	1	1	2	4	12
Vorkommen (Bsp.)	1	0	1	0	1

- Stellvertretend für eine viele Probleme im Unternehmenskontext
(z.B. Logistik, Produktionsplanung, Budgetzuweisung (Projektkosten vs. ROI), Lagerhaltung / Regaloptimierung, ...)

Geeignete Repräsentation des Genoms

Repräsentationsform	Beispiel	Umsetzung
(Sequenz von) Binärwerten	Packproblem (Knapsack)	Jedes Gen wird durch ein einzelnes Bit dargestellt (ggf. ein Gen als Gruppe, die beim Crossover nicht getrennt werden darf), z.B. (0, 1, 0, 1, 0) für: 1. und 3. Element im Rucksack, andere nicht
(Sequenz von) Integers	Zuordnungsprobleme	Ein Gen darf nur ganzzahlige Werte annehmen, ggf. mit eingeschränktem Wertebereich, z.B. aus {0, 1, 2, 3}, {North, West, South, East}, Bsp.-Genom: (0, 1, 0, 2, 3, 3) für Einteilung/Zuordnung von 6 Personen in 4 Gruppen
(Sequenz von) Floating Points	Funktionsmaximierung, BoxCar2D	Ein Gen darf ganzzahlige Werte annehmen, wenn etwa eine genauere Lösung gefragt ist, das Genom ist also eine Sequenz von Floats: (2.5, 4.25, ...)
Permutation	Traveling Salesman Problem, Scheduling von Jobs	Sinnvoll, wenn eine bestimmte Reihenfolge gesucht ist, das Genom sieht dann z.B. so aus: (5, 2, 3, 1, 4)
(Sequenz von) Tupeln	Scheduling Probleme	<i>Beispiel:</i> Ein Tupel kodiert die Zuordnung eines Jobs zu einem Worker (Integer) plus dessen Priorität (Floating Point) unter den diesem Worker zugewiesenen Job, wobei eine niedrigere Zahl z.B. bedeutet, dass der Job zuerst erledigt werden muss: ((W05, 1.5), (W02, 3.1), (W05, 1), ...) bedeutet: Job 0 wird an Worker 05 gegeben, ebenso wie Job 2. Job 2 soll vor Job 0 ausgeführt werden usw.

Beispiel: Scheduling I



- Gegeben ist eine Menge von **Aufträgen**, die von einer Flotte von **Trucks** transportiert werden sollen. Jeder Auftrag besitzt eine **Lademenge**. Jeder Truck hat eine **begrenzte Kapazität**.
- Ziel ist es, jedem Auftrag **genau einen Truck** zuzuweisen und eine **Reihenfolge der Aufträge pro Truck** festzulegen. Dabei darf die **Summe der Lademengen der einem Truck zugewiesenen Aufträge dessen Kapazität nicht überschreiten**.
- Gesucht ist ein **gültiger Schedule**, der alle Aufträge auf die Trucks verteilt und eine Reihenfolge der Ausführung bestimmt. Optional kann zusätzlich ein **Optimierungsziel** verfolgt werden, z. B. Minimierung der Gesamttransportzeit oder Maximierung der Auslastung der Trucks.

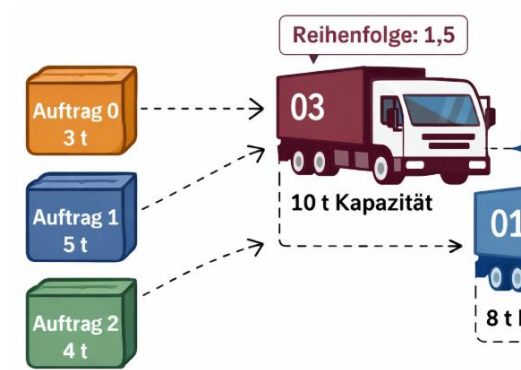
Beispiel: Scheduling II

Genom:

- Ein Tupel kodiert die **Zuordnung eines Auftrags zu einem Truck (Integer)** sowie dessen **Priorität bzw. Reihenfolge (Floating Point)** unter den diesem Truck zugewiesenen Aufträgen.
- Zusätzlich besitzt jeder Auftrag eine **Lademenge**, und jeder Truck eine **maximale Kapazität**.

Interpretation:

- Jeder Auftrag darf **genau einem Truck** zugewiesen werden.
- Die **Summe der Lademengen** der Aufträge auf einem Truck darf dessen **Kapazität nicht überschreiten**.
- Die **Priorität bestimmt die Ausführungsreihenfolge** der Aufträge auf diesem Truck (kleinere Zahl = früher ausgeführt).



Beispiel-Lösung:

(T03, 1.2) , (T01, 2.7) , (T03, 0.8) , (T02, 1.5)

d.h.

Auftrag 0 → Truck 03, Reihenfolge 1.2

Auftrag 1 → Truck 01, Reihenfolge 2.7

Auftrag 2 → Truck 03, Reihenfolge 0.8

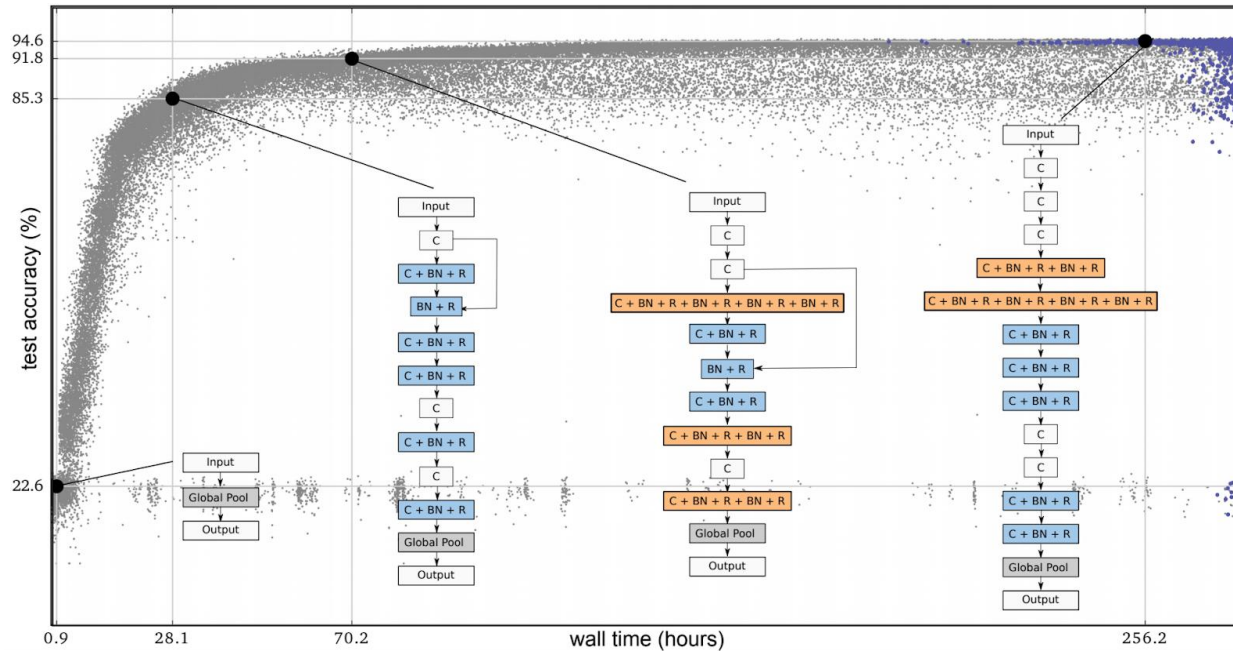
Auftrag 3 → Truck 02, Reihenfolge 1.5

→ Auf **Truck 03** wird Auftrag 2 vor Auftrag 0 ausgeführt.

Noch zu tun: Kapazitätsprüfung und ggf. hohe Penalty.

Einige Praxisbeispiele

Neural Architecture Search



Google Brain team, 2018: [Using Evolutionary AutoML to Discover Neural Network Architectures](#)



Public domain images



Einige Praxisbeispiele

Evolutionary F1 Race Strategy (welche Reifenmischung wann?)

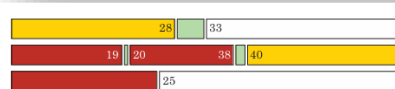


Figure 5: The GA strategies for Barcelona.



Figure 6: Pirelli strategies for Barcelona.

Red indicates the Soft compound;
yellow indicates the Medium compound; white indicates the Hard compound

w/ Pirelli, 2023: [Proc. of the Companion Conf. on Genetic and Evolutionary Computation](#)

Optimierung von Flugzeugturbinen



- Coarse optimization - 100 design variables
- Hybrid GA + numerical optimization + expert system
- Found 2% increase in efficiency
- Spending \$250K to test in laboratory
- Boeing 777 design based on these results

Source: [Artificial intelligence 1: informed search](#)

General Electric, Powell et al, 1989

Evolutionäre Algorithmen vs. Heuristische Suche

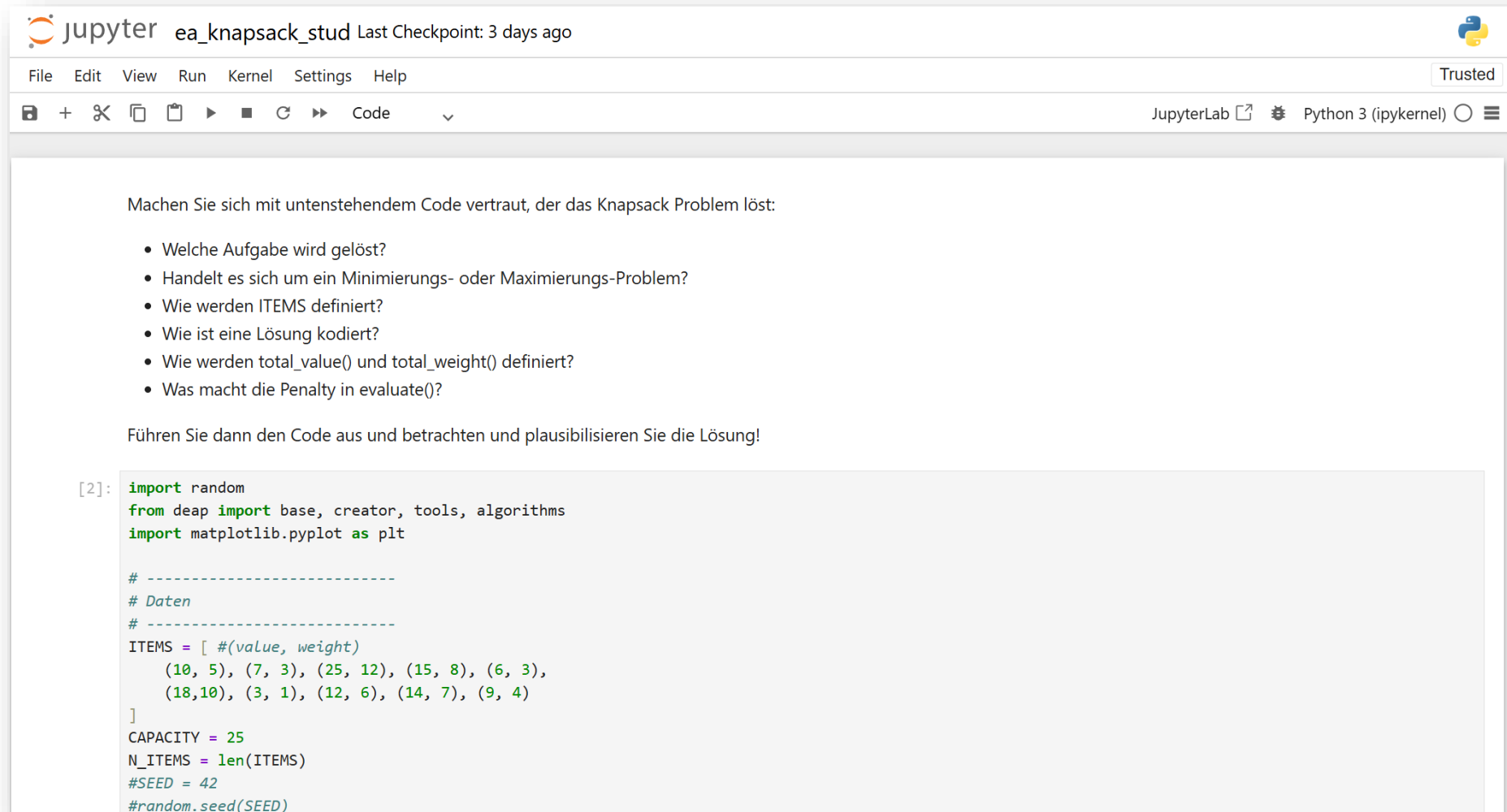
EA:

- + **Globale** Suche, robust gegen lokale Optima
- + Gut **parallelisierbar**
- + **Flexibel** auf viele Problemtypen anwendbar
- + Funktionieren ohne detailliertes **Domänenwissen** (aber auch nicht ohne)
- Hoher **Rechenaufwand** (viele Fitnessauswertungen)
- **Keine Garantie** für Optimum
- Sehr **sensitiv** bzgl. Parameterwahl (z.B. Mutations- und Crossoverrate)
- Gefahr der **vorzeitigen** Konvergenz

Heuristische Suche:

- + Nutzt **Domänenwissen** für gezielte Suche
- + Oft **sehr schnell** bei guter Heuristik
- + **Optimalitätsaussagen** möglich (z.B. für A* mit zulässiger Heuristik)
- + **Deterministisches** Verhalten
- Braucht gute Heuristik (nicht immer verfügbar)
- Weniger **flexibel** auf andere Problemtypen
- Teilweise **komplexe** Heuristikkonstruktion

Übungsaufgabe: Knapsack



The screenshot shows a JupyterLab window titled "ea_knapsack_stud" with a "Last Checkpoint: 3 days ago" status. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and execution. The main area contains a code cell with the following Python code:

```
[2]: import random
from deap import base, creator, tools, algorithms
import matplotlib.pyplot as plt

# -----
# Daten
# -----
ITEMS = [ #(value, weight)
          (10, 5), (7, 3), (25, 12), (15, 8), (6, 3),
          (18, 10), (3, 1), (12, 6), (14, 7), (9, 4)
        ]
CAPACITY = 25
N_ITEMS = len(ITEMS)
#SEED = 42
#random.seed(SEED)
```

Zusammenfassung

- **Evolutionäre Algorithmen**
- **Genetische Operatoren:** Crossover, Mutation, Selektion
- **DEAP Framework**
- Anwendungsbeispiele

